# HW02

ENAE 641: Linear System Dynamics

October 13, 2026 at 23:59:00

*Dr. Robert Sanner*

**Vai Srivastava**

## Problem 1

For a time-invariant linear system with:

$$\boldsymbol{A} = \begin{bmatrix} 0 & 0 & -13 \\ 1 & 0 & -17 \\ 0 & 1 & -5 \end{bmatrix} \quad \boldsymbol{B} = \begin{bmatrix} 13 & 1 \\ 4 & 1 \\ 1 & 0 \end{bmatrix} \quad \boldsymbol{C} = \begin{bmatrix} 1 & -1 & 1 \\ 0 & 1 & -1 \end{bmatrix}$$

and $\boldsymbol{D} = \mathbf{0}$:

a) Determine the representation of the state dynamics in the real, block-modal form, and identify the required coordinate transformation matrix $\boldsymbol{P}$.

b) Give a physically meaningful (i.e. real valued) initial state $\mathbf{x}_0$ which will excite **only** the oscillatory mode of the homogeneous state dynamics. Repeat for the purely exponential mode.

c) Explicitly compute the transfer function matrix $\boldsymbol{G}(s)$ using the transformed state-space model in Part A, and verify that the result is identical to that obtained using the original state-space model.

a) Real block-modal form

$\boldsymbol{A}$ is a companion matrix, so its characteristic polynomial can be read off its last column:

$$\det(s\boldsymbol{I} - \boldsymbol{A}) = s^3 + 5s^2 + 17s + 13 = (s+1)(s^2 + 4s + 13)$$

The eigenvalues are $\lambda_{1,2} = -2 \pm 3j$ (the oscillatory mode) and $\lambda_3 = -1$ (the purely exponential mode).

For an eigenvalue $\lambda$, the rows of $(\boldsymbol{A} - \lambda\boldsymbol{I})\mathbf{v} = \mathbf{0}$ are

$$-\lambda v_1 - 13v_3 = 0, \quad v_1 - \lambda v_2 - 17v_3 = 0, \quad v_2 - (\lambda + 5)v_3 = 0$$

Setting $v_3 = 1$, the last two rows give $v_2 = \lambda + 5$ and $v_1 = \lambda^2 + 5\lambda + 17$. The first row then reads $-(\lambda^3 + 5\lambda^2 + 17\lambda + 13) = 0$, which holds because $\lambda$ is a root of the characteristic polynomial. Thus

$$\mathbf{v}(\lambda) = \begin{bmatrix} \lambda^2 + 5\lambda + 17 \\ \lambda + 5 \\ 1 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v}_1 = \mathbf{v}(-2 + 3j) = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} + j \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \mathbf{v}(-1) = \begin{bmatrix} 13 \\ 4 \\ 1 \end{bmatrix}$$

Write $\mathbf{v}_1 = \mathbf{a} + j\mathbf{b}$ and $\lambda_1 = \sigma + j\omega$ with $\sigma = -2$, $\omega = 3$. The real and imaginary parts of $\boldsymbol{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ are

$$\boldsymbol{A}\mathbf{a} = \sigma\mathbf{a} - \omega\mathbf{b}, \quad \boldsymbol{A}\mathbf{b} = \omega\mathbf{a} + \sigma\mathbf{b} \quad \Rightarrow \quad \boldsymbol{A}[\mathbf{a}\ \mathbf{b}] = [\mathbf{a}\ \mathbf{b}] \begin{bmatrix} \sigma & \omega \\ -\omega & \sigma \end{bmatrix}$$

so $\mathbf{a}$ and $\mathbf{b}$ span a real invariant subspace carrying the oscillatory mode. Placing the complex block in the upper left, the coordinate transformation $\mathbf{x} = \boldsymbol{P}\mathbf{z}$ uses

$$\boldsymbol{P} = [\mathrm{Re}(\mathbf{v}_1)\ \ \mathrm{Im}(\mathbf{v}_1)\ \ \mathbf{v}_3] = \begin{bmatrix} 2 & 3 & 13 \\ 3 & 3 & 4 \\ 1 & 0 & 1 \end{bmatrix} \quad \blacksquare$$

with $\det(\boldsymbol{P}) = -30$ and

$$\boldsymbol{P}^{-1} = \frac{1}{30} \begin{bmatrix} -3 & 3 & 27 \\ -1 & 11 & -31 \\ 3 & -3 & 3 \end{bmatrix}$$

In the new coordinates, $\dot{\mathbf{z}} = \boldsymbol{\Lambda}\mathbf{z} + \bar{\boldsymbol{B}}\mathbf{u}$ and $\mathbf{y} = \bar{\boldsymbol{C}}\mathbf{z}$, with $\bar{\boldsymbol{D}} = \boldsymbol{D} = \mathbf{0}$ and

$$\boldsymbol{\Lambda} = \boldsymbol{P}^{-1}\boldsymbol{A}\boldsymbol{P} = \begin{bmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \quad \blacksquare$$

$$\bar{\boldsymbol{B}} = \boldsymbol{P}^{-1}\boldsymbol{B} = \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{3} \\ 1 & 0 \end{bmatrix}, \quad \bar{\boldsymbol{C}} = \boldsymbol{C}\boldsymbol{P} = \begin{bmatrix} 0 & 0 & 10 \\ 2 & 3 & 3 \end{bmatrix} \quad \blacksquare$$

The Python calculation confirms these matrices:

```
characteristic polynomial: s^3 + 5 s^2 + 17 s + 13
eigenvalues = [-2.-3.j -2.+3.j -1.+0.j]
v(-2 + 3j) = [2.+3.j 3.+3.j 1.+0.j]
v(-1) = [13.  4.  1.]

P =
[[ 2.  3. 13.]
 [ 3.  3.  4.]
 [ 1.  0.  1.]]

30 P^-1 =
[[ -3.   3.  27.]
 [ -1.  11. -31.]
 [  3.  -3.   3.]]

Lambda = P^-1 A P =
[[-2.  3.  0.]
 [-3. -2.  0.]
 [ 0.  0. -1.]]

B_bar = P^-1 B =
[[0.     0.    ]
 [0.     0.3333]
 [1.     0.    ]]

C_bar = C P =
[[ 0.  0. 10.]
 [ 2.  3.  3.]]
```

b) Single-mode initial states

The homogeneous response is $\mathbf{x}(t) = e^{\boldsymbol{A}t}\mathbf{x}_0 = \boldsymbol{P}e^{\boldsymbol{\Lambda}t}\mathbf{z}_0$ with $\mathbf{z}_0 = \boldsymbol{P}^{-1}\mathbf{x}_0$, where

$$e^{\boldsymbol{\Lambda}t} = \begin{bmatrix} e^{-2t}\cos(3t) & e^{-2t}\sin(3t) & 0 \\ -e^{-2t}\sin(3t) & e^{-2t}\cos(3t) & 0 \\ 0 & 0 & e^{-t} \end{bmatrix}$$

Since $e^{\boldsymbol{\Lambda} t}$ is block diagonal, a mode is excited only if $\mathbf{z}_0$ has a nonzero component in that mode's block. Any real, nonzero $\mathbf{x}_0 = \alpha\mathbf{p}_1 + \beta\mathbf{p}_2$ has $z_3(0) = 0$ and excites only the oscillatory mode. Choosing $\mathbf{x}_0 = \mathbf{p}_1$ (so $\mathbf{z}_0 = [1, 0, 0]^\mathsf{T}$):

$$\mathbf{x}(t) = e^{-2t}(\cos(3t)\mathbf{p}_1 - \sin(3t)\mathbf{p}_2) = e^{-2t}\begin{bmatrix} 2\cos(3t) - 3\sin(3t) \\ 3\cos(3t) - 3\sin(3t) \\ \cos(3t) \end{bmatrix}$$

$$\mathbf{x}_0 = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} \quad \blacksquare$$

Similarly, any nonzero real multiple of $\mathbf{p}_3$ has $z_1(0) = z_2(0) = 0$ and excites only the exponential mode. Choosing $\mathbf{x}_0 = \mathbf{p}_3$ (so $\mathbf{z}_0 = [0, 0, 1]^\mathsf{T}$):

$$\mathbf{x}(t) = e^{-t}\begin{bmatrix} 13 \\ 4 \\ 1 \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} 13 \\ 4 \\ 1 \end{bmatrix} \quad \blacksquare$$

Simulating both homogeneous responses confirms that the unexcited modal coordinates stay at zero:

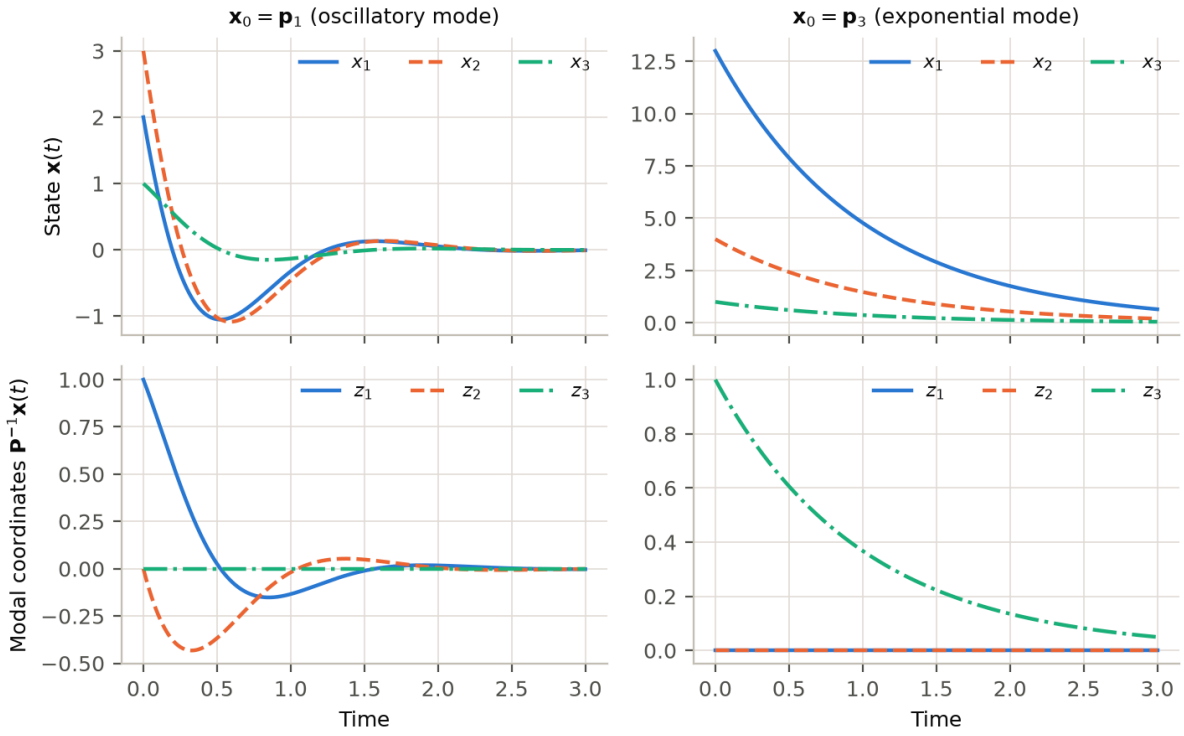



Figure 1: Homogeneous response (top) and modal coordinates $\mathbf{z} = \boldsymbol{P}^{-1}\mathbf{x}$ (bottom) for each single-mode initial state.

```
x0 = p1: max |z_3(t)| = 2.4e-16, max |x(t) - e^(-2t)(cos(3t) p1 - sin(3t) p2)| = 7.1e-15
x0 = p3: max |z_1(t)|, |z_2(t)| = 3.2e-15, max |x(t) - e^(-t) p3| = 7.1e-15
```

c) Transfer function matrix

Since $\bar{\boldsymbol{D}} = \boldsymbol{0}$, the transformed model gives $\boldsymbol{G}(s) = \bar{\boldsymbol{C}}(s\boldsymbol{I} - \boldsymbol{\Lambda})^{-1}\bar{\boldsymbol{B}}$. The matrix $s\boldsymbol{I} - \boldsymbol{\Lambda}$ is block diagonal, so it can be inverted block by block:

$$\begin{bmatrix} s+2 & -3 \\ 3 & s+2 \end{bmatrix}^{-1} = \frac{1}{s^2+4s+13}\begin{bmatrix} s+2 & 3 \\ -3 & s+2 \end{bmatrix}, \quad (s+1)^{-1} = \frac{1}{s+1}$$

Then

$$(s\boldsymbol{I} - \boldsymbol{\Lambda})^{-1}\bar{\boldsymbol{B}} = \begin{bmatrix} \frac{s+2}{s^2+4s+13} & \frac{3}{s^2+4s+13} & 0 \\ -\frac{3}{s^2+4s+13} & \frac{s+2}{s^2+4s+13} & 0 \\ 0 & 0 & \frac{1}{s+1} \end{bmatrix}\begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{3} \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{s^2+4s+13} \\ 0 & \frac{s+2}{3(s^2+4s+13)} \\ \frac{1}{s+1} & 0 \end{bmatrix}$$

and premultiplying by $\bar{\boldsymbol{C}}$:

$$\bar{\boldsymbol{C}}(s\boldsymbol{I} - \boldsymbol{\Lambda})^{-1}\bar{\boldsymbol{B}} = \begin{bmatrix} 0 & 0 & 10 \\ 2 & 3 & 3 \end{bmatrix}\begin{bmatrix} 0 & \frac{1}{s^2+4s+13} \\ 0 & \frac{s+2}{3(s^2+4s+13)} \\ \frac{1}{s+1} & 0 \end{bmatrix} = \begin{bmatrix} \frac{10}{s+1} & 0 \\ \frac{3}{s+1} & \frac{2+(s+2)}{s^2+4s+13} \end{bmatrix}$$

$$\boldsymbol{G}(s) = \begin{bmatrix} \frac{10}{s+1} & 0 \\ \frac{3}{s+1} & \frac{s+4}{s^2+4s+13} \end{bmatrix} \quad \blacksquare$$

The modal form makes the structure visible. $\bar{\boldsymbol{B}}$ routes $u_1$ only into the exponential mode $z_3$ and $u_2$ only into the oscillatory mode, and $y_1$ observes only $z_3$ (the first row of $\bar{\boldsymbol{C}}$ is $[0, 0, 10]$). Hence $G_{12}(s) = 0$, and the oscillatory poles do not appear in the first column.

With the original model, every entry of $\boldsymbol{C}(s\boldsymbol{I} - \boldsymbol{A})^{-1}\boldsymbol{B}$ has the common denominator $\det(s\boldsymbol{I} - \boldsymbol{A}) = (s+1)(s^2+4s+13)$. Using `ss` and `tf` from python-control (the Python counterparts of the Matlab commands) gives the numerators below; factoring them:

$$G_{11}(s) = \frac{10s^2+40s+130}{(s+1)(s^2+4s+13)} = \frac{10(s^2+4s+13)}{(s+1)(s^2+4s+13)} = \frac{10}{s+1}$$

$$G_{21}(s) = \frac{3s^2+12s+39}{(s+1)(s^2+4s+13)} = \frac{3(s^2+4s+13)}{(s+1)(s^2+4s+13)} = \frac{3}{s+1}$$

$$G_{22}(s) = \frac{s^2+5s+4}{(s+1)(s^2+4s+13)} = \frac{(s+1)(s+4)}{(s+1)(s^2+4s+13)} = \frac{s+4}{s^2+4s+13}$$

and $G_{12}(s) = 0$. After the pole-zero cancellations, this is identical to the modal-form result. This is expected, since a transfer function is invariant under a similarity transformation:

$$\bar{\boldsymbol{C}}(s\boldsymbol{I} - \boldsymbol{\Lambda})^{-1}\bar{\boldsymbol{B}} = \boldsymbol{C}\boldsymbol{P}\big(\boldsymbol{P}^{-1}(s\boldsymbol{I} - \boldsymbol{A})\boldsymbol{P}\big)^{-1}\boldsymbol{P}^{-1}\boldsymbol{B} = \boldsymbol{C}(s\boldsymbol{I} - \boldsymbol{A})^{-1}\boldsymbol{B}$$

The Python calculation, which also compares both models and the hand result along $s = j\omega$:

```
original model, C (sI - A)^-1 B before cancellation:
  G11(s) = (10 s^2 + 40 s + 130) / (s^3 + 5 s^2 + 17 s + 13)
  G12(s) = 0
  G21(s) = (3 s^2 + 12 s + 39) / (s^3 + 5 s^2 + 17 s + 13)
  G22(s) = (s^2 + 5 s + 4) / (s^3 + 5 s^2 + 17 s + 13)
original model after pole-zero cancellation:
  G11(s) = 10 / (s + 1)
  G12(s) = 0
```

```
    G21(s) = 3 / (s + 1)
   G22(s) = (s + 4) / (s^2 + 4 s + 13)
modal model after pole-zero cancellation:
   G11(s) = 10 / (s + 1)
   G12(s) = 0
   G21(s) = 3 / (s + 1)
   G22(s) = (s + 4) / (s^2 + 4 s + 13)
max |G_original(jw) - G_modal(jw)| = 5.4e-15
max |G_original(jw) - G_hand(jw)| = 5.3e-15
```

## Problem 2

The homogeneous state equation for a linear system has matrix $\boldsymbol{A}(t)$ that can be partitioned as

$$\begin{bmatrix} \boldsymbol{A}_1(t) & \boldsymbol{A}_{1,2}(t) \\ 0 & \boldsymbol{A}_2(t) \end{bmatrix}$$

$\boldsymbol{A}_1$ and $\boldsymbol{A}_2$ are square, but of possibly different sizes, while $\boldsymbol{A}_{1,2}$ is not necessarily square.

a) Show that the state transition matrix has the compatible form:

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} \boldsymbol{\Phi}_1(t,\tau) & \boldsymbol{\Phi}_{1,2}(t,\tau) \\ 0 & \boldsymbol{\Phi}_2(t,\tau) \end{bmatrix}$$

where $\boldsymbol{\Phi}_1$ is the STM for $\boldsymbol{A}_1(t)$ and $\boldsymbol{\Phi}_2$ is the STM for $\boldsymbol{A}_2(t)$

b) Give an expression for $\boldsymbol{\Phi}_{1,2}(t,\tau)$ in terms of $\boldsymbol{\Phi}_1$, $\boldsymbol{\Phi}_2$, and $\boldsymbol{A}_{1,2}$.

NOTE: $\boldsymbol{\Phi}_{1,2}$ will not be a STM for $\boldsymbol{A}_{1,2}$, since that matrix is not necessarily square.

a) Block-triangular form of the STM

The STM is the unique solution of

$$\frac{\partial}{\partial t}\boldsymbol{\Phi}(t,\tau) = \boldsymbol{A}(t)\boldsymbol{\Phi}(t,\tau), \quad \boldsymbol{\Phi}(\tau,\tau) = \boldsymbol{I}$$

Partition $\boldsymbol{\Phi}$ conformally with $\boldsymbol{A}$, writing its blocks as $\boldsymbol{X}_{ij}(t,\tau)$, so that $\boldsymbol{X}_{11}$ has the size of $\boldsymbol{A}_1$ and $\boldsymbol{X}_{22}$ has the size of $\boldsymbol{A}_2$:

$$\frac{\partial}{\partial t}\begin{bmatrix} \boldsymbol{X}_{11} & \boldsymbol{X}_{12} \\ \boldsymbol{X}_{21} & \boldsymbol{X}_{22} \end{bmatrix} = \begin{bmatrix} \boldsymbol{A}_1 & \boldsymbol{A}_{1,2} \\ \boldsymbol{0} & \boldsymbol{A}_2 \end{bmatrix}\begin{bmatrix} \boldsymbol{X}_{11} & \boldsymbol{X}_{12} \\ \boldsymbol{X}_{21} & \boldsymbol{X}_{22} \end{bmatrix}, \quad \begin{bmatrix} \boldsymbol{X}_{11} & \boldsymbol{X}_{12} \\ \boldsymbol{X}_{21} & \boldsymbol{X}_{22} \end{bmatrix}(\tau,\tau) = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix}$$

Multiplying out the blocks gives four equations:

$$\begin{aligned} \frac{\partial \boldsymbol{X}_{21}}{\partial t} &= \boldsymbol{A}_2(t)\boldsymbol{X}_{21}, & \boldsymbol{X}_{21}(\tau,\tau) &= \boldsymbol{0} \\ \frac{\partial \boldsymbol{X}_{22}}{\partial t} &= \boldsymbol{A}_2(t)\boldsymbol{X}_{22}, & \boldsymbol{X}_{22}(\tau,\tau) &= \boldsymbol{I} \\ \frac{\partial \boldsymbol{X}_{11}}{\partial t} &= \boldsymbol{A}_1(t)\boldsymbol{X}_{11} + \boldsymbol{A}_{1,2}(t)\boldsymbol{X}_{21}, & \boldsymbol{X}_{11}(\tau,\tau) &= \boldsymbol{I} \\ \frac{\partial \boldsymbol{X}_{12}}{\partial t} &= \boldsymbol{A}_1(t)\boldsymbol{X}_{12} + \boldsymbol{A}_{1,2}(t)\boldsymbol{X}_{22}, & \boldsymbol{X}_{12}(\tau,\tau) &= \boldsymbol{0} \end{aligned}$$

The first two equations do not involve the top blocks:

- The equation for $\boldsymbol{X}_{21}$ is linear and homogeneous with a zero initial value. By uniqueness, $\boldsymbol{X}_{21}(t,\tau) = \boldsymbol{\Phi}_2(t,\tau)\boldsymbol{X}_{21}(\tau,\tau) = \boldsymbol{0}$ for all $t$.
- The equation for $\boldsymbol{X}_{22}$ is exactly the defining problem for the STM of $\boldsymbol{A}_2(t)$, so $\boldsymbol{X}_{22} = \boldsymbol{\Phi}_2(t,\tau)$.
- With $\boldsymbol{X}_{21} = \boldsymbol{0}$, the equation for $\boldsymbol{X}_{11}$ reduces to $\frac{\partial \boldsymbol{X}_{11}}{\partial t} = \boldsymbol{A}_1(t)\boldsymbol{X}_{11}$ with $\boldsymbol{X}_{11}(\tau,\tau) = \boldsymbol{I}$. This is the defining problem for the STM of $\boldsymbol{A}_1(t)$, so $\boldsymbol{X}_{11} = \boldsymbol{\Phi}_1(t,\tau)$.

Naming the remaining block $\boldsymbol{\Phi}_{1,2} \coloneqq \boldsymbol{X}_{12}$:

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} \boldsymbol{\Phi}_1(t,\tau) & \boldsymbol{\Phi}_{1,2}(t,\tau) \\ \boldsymbol{0} & \boldsymbol{\Phi}_2(t,\tau) \end{bmatrix} \quad \blacksquare$$

b) Off-diagonal block

With $\boldsymbol{X}_{22} = \boldsymbol{\Phi}_2$, the last block equation is a forced linear equation for $\boldsymbol{\Phi}_{1,2}$. Its dynamics are governed by $\boldsymbol{A}_1$, and its "input" is $\boldsymbol{A}_{1,2}(t)\boldsymbol{\Phi}_2(t,\tau)$:

$$\frac{\partial \boldsymbol{\Phi}_{1,2}}{\partial t} = \boldsymbol{A}_1(t)\boldsymbol{\Phi}_{1,2} + \boldsymbol{A}_{1,2}(t)\boldsymbol{\Phi}_2(t,\tau), \quad \boldsymbol{\Phi}_{1,2}(\tau,\tau) = \boldsymbol{0}$$

Apply the variation-of-constants formula column by column. The homogeneous term $\boldsymbol{\Phi}_1(t,\tau)\boldsymbol{\Phi}_{1,2}(\tau,\tau)$ vanishes, leaving

$$\boldsymbol{\Phi}_{1,2}(t,\tau) = \int_\tau^t \boldsymbol{\Phi}_1(t,\sigma)\boldsymbol{A}_{1,2}(\sigma)\boldsymbol{\Phi}_2(\sigma,\tau)\,\mathrm{d}\sigma \quad \blacksquare$$

Check: at $t = \tau$ the integral vanishes. By the Leibniz rule, with $\frac{\partial}{\partial t}\boldsymbol{\Phi}_1(t,\sigma) = \boldsymbol{A}_1(t)\boldsymbol{\Phi}_1(t,\sigma)$ and $\boldsymbol{\Phi}_1(t,t) = \boldsymbol{I}$:

$$\begin{aligned}\frac{\partial \boldsymbol{\Phi}_{1,2}}{\partial t} &= \boldsymbol{\Phi}_1(t,t)\boldsymbol{A}_{1,2}(t)\boldsymbol{\Phi}_2(t,\tau) + \int_\tau^t \boldsymbol{A}_1(t)\boldsymbol{\Phi}_1(t,\sigma)\boldsymbol{A}_{1,2}(\sigma)\boldsymbol{\Phi}_2(\sigma,\tau)\,\mathrm{d}\sigma \\ &= \boldsymbol{A}_{1,2}(t)\boldsymbol{\Phi}_2(t,\tau) + \boldsymbol{A}_1(t)\boldsymbol{\Phi}_{1,2}(t,\tau)\end{aligned}$$

as required. If $\boldsymbol{A}_1$ is $n_1 \times n_1$ and $\boldsymbol{A}_2$ is $n_2 \times n_2$, the integrand is $(n_1 \times n_1)(n_1 \times n_2)(n_2 \times n_2) = n_1 \times n_2$, so the formula is consistent even though $\boldsymbol{A}_{1,2}$ need not be square. A numerical check on a sample system with a $2 \times 1$ coupling block, comparing the integrated STM of the full system with the formula (Python):

```
sample system: A_1(t) is 2x2, A_12(t) is 2x1, A_2(t) is 1x1, tau = 0.5, t in [0.5, 3.5]
max |Phi_21| = 0.0e+00
max |Phi_11 - Phi_1| = 4.0e-12
max |Phi_22 - Phi_2| = 6.1e-12
max |Phi_12 - integral formula| = 1.5e-11
```

## Problem 3

Given the state-space model:

$$\begin{aligned}\dot{x}_1(t) &= tx_2(t)\\ \dot{x}_2(t) &= -x_2(t) + e^{-t}u(t)\\ y(t) &= tx_1(t)\end{aligned}$$

a) Find the state transition matrix $\Phi(t,\tau)$.

b) Find the impulse response $G(t,\tau)$.

c) Find the response $y(t)$ for $t \geq 1$ when $\boldsymbol{x}(1)^\mathsf{T} = [1,-2]$ and $u(t) = 1$ for all $t \geq 1$.

a) State transition matrix

In matrix form, $\dot{\boldsymbol{x}} = A(t)\boldsymbol{x} + B(t)u$ and $y = C(t)\boldsymbol{x}$ with

$$A(t) = \begin{bmatrix}0 & t\\ 0 & -1\end{bmatrix}, \quad B(t) = \begin{bmatrix}0\\ e^{-t}\end{bmatrix}, \quad C(t) = [t\ \ 0]$$

$A(t)$ is upper triangular, so the homogeneous equations can be solved from the bottom up, starting from $\boldsymbol{x}(\tau)$:

$$\dot{x}_2 = -x_2 \quad \Rightarrow \quad x_2(t) = e^{-(t-\tau)}x_2(\tau)$$

$$\dot{x}_1 = tx_2(t) = te^{-(t-\tau)}x_2(\tau) \quad \Rightarrow \quad x_1(t) = x_1(\tau) + x_2(\tau)\int_\tau^t \sigma e^{-(\sigma-\tau)}\,\mathrm{d}\sigma$$

Using $\int \sigma e^{-\sigma}\,\mathrm{d}\sigma = -(\sigma+1)e^{-\sigma}$:

$$\int_\tau^t \sigma e^{-(\sigma-\tau)}\,\mathrm{d}\sigma = e^{\tau}\left[-(\sigma+1)e^{-\sigma}\right]_\tau^t = (\tau+1) - (t+1)e^{-(t-\tau)}$$

The columns of $\Phi(t,\tau)$ are the responses to $\boldsymbol{x}(\tau) = [1,0]^\mathsf{T}$ and $\boldsymbol{x}(\tau) = [0,1]^\mathsf{T}$:

$$\Phi(t,\tau) = \begin{bmatrix}1 & (\tau+1) - (t+1)e^{-(t-\tau)}\\ 0 & e^{-(t-\tau)}\end{bmatrix} \quad \blacksquare$$

Check: $\Phi(\tau,\tau) = \boldsymbol{I}$, and $\frac{\partial \Phi_{12}}{\partial t} = te^{-(t-\tau)} = t\Phi_{22}$, $\frac{\partial \Phi_{22}}{\partial t} = -\Phi_{22}$, as required by $\frac{\partial \Phi}{\partial t} = A(t)\Phi$. This is also the scalar-block case of Problem 2 with $\Phi_1 = 1$, $\Phi_2 = e^{-(t-\tau)}$, and $A_{1,2} = t$. Compared with the numerically integrated STM (Python):

```
1  max relative error of Phi(t, tau) vs numerical STM = 9.9e-11
```

b) Impulse response

With $D = 0$, the impulse response is $G(t,\tau) = C(t)\Phi(t,\tau)B(\tau)$ for $t \geq \tau$, and zero for $t < \tau$:

$$C(t)\Phi(t,\tau)B(\tau) = [t\ \ 0]\begin{bmatrix}1 & (\tau+1) - (t+1)e^{-(t-\tau)}\\ 0 & e^{-(t-\tau)}\end{bmatrix}\begin{bmatrix}0\\ e^{-\tau}\end{bmatrix} = te^{-\tau}\left[(\tau+1) - (t+1)e^{-(t-\tau)}\right]$$

$$G(t,\tau) = \begin{cases}t[(\tau+1)e^{-\tau} - (t+1)e^{-t}] & t \geq \tau\\ 0 & t < \tau\end{cases} \quad \blacksquare$$

An impulse at $\tau$ sets $\boldsymbol{x}(\tau^+) = B(\tau)$, so simulating the homogeneous system from that state reproduces $G(t,\tau)$ (Python):

```
1  max |simulated impulse response - G(t, tau)| for tau = 0, 1, 2.5: 3.1e-11
```

c) Forced response from $t = 1$

By superposition of the zero-input and zero-state responses,

$$y(t) = C(t)\Phi(t,1)\boldsymbol{x}(1) + \int_1^t G(t,\sigma)u(\sigma)\,\mathrm{d}\sigma$$

*Zero-input part.* With $\boldsymbol{x}(1) = [1,-2]^{\mathsf{T}}$:

$$\Phi(t,1)\boldsymbol{x}(1) = \begin{bmatrix} 1 - 2\big[2 - (t+1)e^{-(t-1)}\big] \\ -2e^{-(t-1)} \end{bmatrix} = \begin{bmatrix} -3 + 2(t+1)e^{1-t} \\ -2e^{1-t} \end{bmatrix} \quad\Rightarrow\quad y_{\text{zi}}(t) = t\big[-3 + 2(t+1)e^{1-t}\big]$$

*Zero-state part.* With $u = 1$ and $\int (\sigma+1)e^{-\sigma}\,\mathrm{d}\sigma = -(\sigma+2)e^{-\sigma}$:

$$\begin{aligned} y_{\text{zs}}(t) &= t\int_1^t \big[(\sigma+1)e^{-\sigma} - (t+1)e^{-t}\big]\,\mathrm{d}\sigma = t\big[3e^{-1} - (t+2)e^{-t} - (t+1)(t-1)e^{-t}\big] \\ &= t\big[3e^{-1} - (t^2+t+1)e^{-t}\big] \end{aligned}$$

Adding the two parts and collecting the $e^{-t}$ terms:

$$y(t) = t\left[\frac{3}{e} - 3 + (2e(t+1) - t^2 - t - 1)e^{-t}\right], \quad t \ge 1 \quad \blacksquare$$

Check: $y(1) = \frac{3}{e} - 3 + \frac{4e-3}{e} = 1 = C(1)\boldsymbol{x}(1)$. As $t \to \infty$, $x_1(t) \to \frac{3}{e} - 3$, so the output grows without bound like $y(t) \approx \big(\frac{3}{e} - 3\big)t \approx -1.896t$ because of the time-varying output gain $C(t) = [t\ \ 0]$. The closed form matches a direct simulation of the state equations (Python):

```
1  y(1) = 1.000000
2  max |y(t) - simulated y(t)| on [1, 10] = 1.1e-10
3  max |y(t) - (C Phi(t, 1) x(1) + integral of G(t, s) ds)| = 7.1e-15
4  y(t) / t -> 3/e - 3 = -1.896362
```

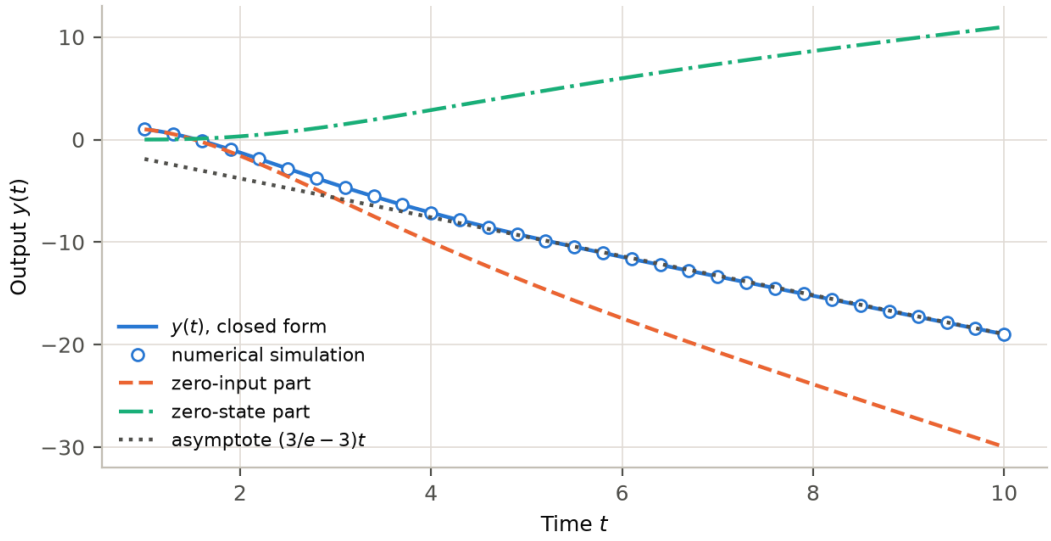



Figure 2: Response $y(t)$ for $\boldsymbol{x}(1)^{\mathsf{T}} = [1, -2]$ and $u = 1$, with its zero-input and zero-state parts.

## Problem 4

The characteristic polynomial of the $\boldsymbol{A}$ matrix for an LTI dynamic system can be written as $r(\lambda) = (\lambda^2 + 2\lambda + 10)(\lambda + 3)^3$. You also know that the rank of $(\boldsymbol{A} + 3\boldsymbol{I})$ is 4.

Let $\boldsymbol{P}$ be the matrix which will transform $\boldsymbol{A}$ to an appropriate real block modal form. Assume the $\boldsymbol{P}$ matrix is such that the components of the modal form arising from the complex eigenvalues will be located in the upper left corner.

a) Compute the matrix $\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\boldsymbol{P}$.

b) Suppose that we identify the outputs of this system to be $\boldsymbol{y}(t) = \boldsymbol{P}^{-1}\mathbf{x}(t)$. If the initial condition is $\mathbf{x}_0 = \mathbf{p}_4$ where $\mathbf{p}_4$ is the fourth column of the $\boldsymbol{P}$ matrix, find $\mathbf{y}(t)$ explicitly as a function of time.

c) Suppose in Part B that the initial condition is instead $\mathbf{x}_0 = \mathbf{p}_2$. Find $\mathbf{y}(t)$.

a) Modal form and $\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\boldsymbol{P}$

$r(\lambda)$ has degree 5, so $\boldsymbol{A}$ is $5 \times 5$. Its eigenvalues are $\lambda = -1 \pm 3j$ (from $\lambda^2 + 2\lambda + 10 = (\lambda + 1)^2 + 9$) and $\lambda = -3$ with algebraic multiplicity 3. The geometric multiplicity of $\lambda = -3$ is

$$\dim \mathcal{N}(\boldsymbol{A} + 3\boldsymbol{I}) = 5 - \operatorname{rank}(\boldsymbol{A} + 3\boldsymbol{I}) = 5 - 4 = 1$$

so $\lambda = -3$ has only one independent eigenvector, and its three-fold eigenvalue forms a single $3 \times 3$ Jordan block. (Two or three blocks would give $\operatorname{rank}(\boldsymbol{A} + 3\boldsymbol{I}) = 3$ or 2.) The columns of $\boldsymbol{P}$ are therefore:

- $\mathbf{p}_1 = \operatorname{Re}(\mathbf{v})$ and $\mathbf{p}_2 = \operatorname{Im}(\mathbf{v})$, where $\mathbf{v}$ is an eigenvector for $-1 + 3j$. As in Problem 1,

$$\boldsymbol{A}[\mathbf{p}_1 \;\; \mathbf{p}_2] = [\mathbf{p}_1 \;\; \mathbf{p}_2]\begin{bmatrix} -1 & 3 \\ -3 & -1 \end{bmatrix}$$

- A chain of generalized eigenvectors for $\lambda = -3$:

$$(\boldsymbol{A} + 3\boldsymbol{I})\mathbf{p}_3 = \mathbf{0}, \quad (\boldsymbol{A} + 3\boldsymbol{I})\mathbf{p}_4 = \mathbf{p}_3, \quad (\boldsymbol{A} + 3\boldsymbol{I})\mathbf{p}_5 = \mathbf{p}_4$$

  Equivalently, $\boldsymbol{A}\mathbf{p}_3 = -3\mathbf{p}_3$, $\boldsymbol{A}\mathbf{p}_4 = \mathbf{p}_3 - 3\mathbf{p}_4$, and $\boldsymbol{A}\mathbf{p}_5 = \mathbf{p}_4 - 3\mathbf{p}_5$.

Together these give the real block-modal (real Jordan) form:

$$\boldsymbol{J} = \boldsymbol{P}^{-1}\boldsymbol{A}\boldsymbol{P} = \begin{bmatrix} -1 & 3 & 0 & 0 & 0 \\ -3 & -1 & 0 & 0 & 0 \\ 0 & 0 & -3 & 1 & 0 \\ 0 & 0 & 0 & -3 & 1 \\ 0 & 0 & 0 & 0 & -3 \end{bmatrix}$$

Since $\boldsymbol{P}^{-1}\boldsymbol{A}^k\boldsymbol{P} = \boldsymbol{J}^k$, the exponential series gives $\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\boldsymbol{P} = e^{\boldsymbol{J}t}$, which can be computed block by block:

- The complex block is $-\boldsymbol{I} + 3\boldsymbol{S}$ with $\boldsymbol{S} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$. These two terms commute, and $\boldsymbol{S}^2 = -\boldsymbol{I}$ gives $e^{\theta\boldsymbol{S}} = \cos(\theta)\boldsymbol{I} + \sin(\theta)\boldsymbol{S}$, so

$$e^{(-\boldsymbol{I}+3\boldsymbol{S})t} = e^{-t}e^{3t\boldsymbol{S}} = e^{-t}\begin{bmatrix} \cos(3t) & \sin(3t) \\ -\sin(3t) & \cos(3t) \end{bmatrix}$$

- The Jordan block is $-3\boldsymbol{I} + \boldsymbol{N}$, where $\boldsymbol{N}$ has ones on the superdiagonal, so $\boldsymbol{N}^3 = \mathbf{0}$. These two terms also commute, so the series for $e^{\boldsymbol{N}t}$ terminates:

$$e^{(-3\boldsymbol{I}+\boldsymbol{N})t} = e^{-3t}\left(\boldsymbol{I} + t\boldsymbol{N} + \frac{t^2}{2}\boldsymbol{N}^2\right) = e^{-3t}\begin{bmatrix} 1 & t & \frac{t^2}{2} \\ 0 & 1 & t \\ 0 & 0 & 1 \end{bmatrix}$$

Thus

$$\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\boldsymbol{P} = e^{\boldsymbol{J}t} = \begin{bmatrix} e^{-t}\cos(3t) & e^{-t}\sin(3t) & 0 & 0 & 0 \\ -e^{-t}\sin(3t) & e^{-t}\cos(3t) & 0 & 0 & 0 \\ 0 & 0 & e^{-3t} & te^{-3t} & \frac{t^2}{2}e^{-3t} \\ 0 & 0 & 0 & e^{-3t} & te^{-3t} \\ 0 & 0 & 0 & 0 & e^{-3t} \end{bmatrix} \quad \blacksquare$$

b) Response from $\mathbf{x}_0 = \mathbf{p}_4$

With $\mathbf{x}(t) = e^{\boldsymbol{A}t}\mathbf{x}_0$:

$$\mathbf{y}(t) = \boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\mathbf{x}_0 = \left(\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}\boldsymbol{P}\right)\boldsymbol{P}^{-1}\mathbf{x}_0 = e^{\boldsymbol{J}t}\boldsymbol{P}^{-1}\mathbf{x}_0$$

Since $\mathbf{p}_4$ is the fourth column of $\boldsymbol{P}$, $\boldsymbol{P}^{-1}\mathbf{p}_4 = \mathbf{e}_4$, so $\mathbf{y}(t)$ is the fourth column of $e^{\boldsymbol{J}t}$:

$$\mathbf{y}(t) = \begin{bmatrix} 0 \\ 0 \\ te^{-3t} \\ e^{-3t} \\ 0 \end{bmatrix} \quad \blacksquare$$

In the original coordinates, $\mathbf{x}(t) = e^{-3t}(t\mathbf{p}_3 + \mathbf{p}_4)$. Starting on the generalized eigenvector $\mathbf{p}_4$ also drives the eigenvector direction $\mathbf{p}_3$ through the secular term $te^{-3t}$. It never excites $\mathbf{p}_5$ or the oscillatory mode.

c) Response from $\mathbf{x}_0 = \mathbf{p}_2$

Now $\boldsymbol{P}^{-1}\mathbf{p}_2 = \mathbf{e}_2$, so $\mathbf{y}(t)$ is the second column of $e^{\boldsymbol{J}t}$:

$$\mathbf{y}(t) = \begin{bmatrix} e^{-t}\sin(3t) \\ e^{-t}\cos(3t) \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \blacksquare$$

In the original coordinates, $\mathbf{x}(t) = e^{-t}(\sin(3t)\mathbf{p}_1 + \cos(3t)\mathbf{p}_2)$, a decaying oscillation confined to the oscillatory mode.

As a numerical check, $\boldsymbol{A} = \boldsymbol{P}\boldsymbol{J}\boldsymbol{P}^{-1}$ was built from a random invertible $\boldsymbol{P}$. The Python calculation confirms that its characteristic polynomial and ranks match the given data, and that all three results above match $\boldsymbol{P}^{-1}e^{\boldsymbol{A}t}$ computed with a matrix exponential:

```
r(lambda) = l^5 + 11 l^4 + 55 l^3 + 171 l^2 + 324 l + 270
charpoly(A) = l^5 + 11 l^4 + 55 l^3 + 171 l^2 + 324 l + 270  (A = P J P^-1, random P)
rank (A + 3I)^k for k = 1, 2, 3: [4, 3, 2]
a) max |P^-1 e^(At) P - closed form| = 1.2e-15
b) max |P^-1 e^(At) p_4 - closed form| = 4.2e-16
c) max |P^-1 e^(At) p_2 - closed form| = 1.0e-15
```

## Problem 5

Suppose the matrix $\boldsymbol{A}(t)$ for a linear system can be written as

$$\boldsymbol{A}(t) = \sum_{j=1}^{r} f_j(t)\boldsymbol{A}_j$$

where the $\boldsymbol{A}_j$ are constant, and additionally $\boldsymbol{A}_i\boldsymbol{A}_j = \boldsymbol{A}_j\boldsymbol{A}_i$ (commutative) for every $i$, $j$.

a) Show that the state transition matrix for this system can be written as a matrix exponential function.

b) Show that the matrix exponential form for the STM can in fact be factored as

$$\boldsymbol{\Phi}(t,\tau) = e^{\boldsymbol{A}_1 \int_\tau^t f_1(\sigma)\,\mathrm{d}\sigma} e^{\boldsymbol{A}_2 \int_\tau^t f_2(\sigma)\,\mathrm{d}\sigma} \ \ldots\ e^{\boldsymbol{A}_r \int_\tau^t f_{r(\sigma)}\,\mathrm{d}\sigma}$$

c) Use the observations above to compute the state transition matrix for

$$\boldsymbol{A}(t) = \begin{bmatrix} \cos(3t) & \sin(3t) \\ -\sin(3t) & \cos(3t) \end{bmatrix}$$

a) STM as a matrix exponential

Define

$$F_j(t,\tau) = \int_\tau^t f_j(\sigma)\,\mathrm{d}\sigma, \quad \boldsymbol{M}(t,\tau) = \int_\tau^t \boldsymbol{A}(\sigma)\,\mathrm{d}\sigma = \sum_{j=1}^{r} F_j(t,\tau)\boldsymbol{A}_j$$

Because the $\boldsymbol{A}_j$ commute pairwise, $\boldsymbol{A}(t)$ commutes with $\boldsymbol{M}(t,\tau)$:

$$\boldsymbol{A}(t)\boldsymbol{M}(t,\tau) = \sum_{i=1}^{r}\sum_{j=1}^{r} f_i(t)F_j(t,\tau)\boldsymbol{A}_i\boldsymbol{A}_j = \sum_{i=1}^{r}\sum_{j=1}^{r} F_j(t,\tau)f_i(t)\boldsymbol{A}_j\boldsymbol{A}_i = \boldsymbol{M}(t,\tau)\boldsymbol{A}(t)$$

Since $\frac{\partial \boldsymbol{M}}{\partial t} = \boldsymbol{A}(t)$ commutes with $\boldsymbol{M}$, the product rule collapses:

$$\frac{\partial}{\partial t}\boldsymbol{M}^k = \sum_{m=0}^{k-1} \boldsymbol{M}^m \boldsymbol{A}(t) \boldsymbol{M}^{k-1-m} = k\boldsymbol{A}(t)\boldsymbol{M}^{k-1}$$

Assume the $f_j$ are continuous, so $\boldsymbol{A}(t)$ and $\boldsymbol{M}(t,\tau)$ are bounded on any bounded $t$-interval. Then the differentiated series $\boldsymbol{A}(t)\sum_{k\geq 1} \frac{\boldsymbol{M}^{k-1}}{(k-1)!}$ converges uniformly there, which justifies differentiating the exponential series term by term:

$$\frac{\partial}{\partial t} e^{\boldsymbol{M}(t,\tau)} = \sum_{k=1}^{\infty} \frac{k}{k!}\boldsymbol{A}(t)\boldsymbol{M}^{k-1} = \boldsymbol{A}(t)\sum_{k=1}^{\infty}\frac{\boldsymbol{M}^{k-1}}{(k-1)!} = \boldsymbol{A}(t)e^{\boldsymbol{M}(t,\tau)}$$

Also $e^{\boldsymbol{M}(\tau,\tau)} = e^{\boldsymbol{0}} = \boldsymbol{I}$. So $e^{\boldsymbol{M}(t,\tau)}$ solves the defining problem for the STM, whose solution is unique:

$$\boldsymbol{\Phi}(t,\tau) = e^{\int_\tau^t \boldsymbol{A}(\sigma)\,\mathrm{d}\sigma} = \exp\left(\sum_{j=1}^{r} \boldsymbol{A}_j \int_\tau^t f_j(\sigma)\,\mathrm{d}\sigma\right) \quad \blacksquare$$

Without commutativity, the derivative of $\boldsymbol{M}^k$ does not collapse to $k\boldsymbol{A}\boldsymbol{M}^{k-1}$, and this formula generally fails (see Problem 6, Part C).

b) Factoring the exponential

Let $\boldsymbol{X}_j = F_j(t,\tau)\boldsymbol{A}_j$. These commute pairwise, since $\boldsymbol{X}_i\boldsymbol{X}_j = F_iF_j\boldsymbol{A}_i\boldsymbol{A}_j = F_jF_i\boldsymbol{A}_j\boldsymbol{A}_i = \boldsymbol{X}_j\boldsymbol{X}_i$.

For commuting $\boldsymbol{X}$ and $\boldsymbol{Y}$, the binomial theorem holds: $(\boldsymbol{X}+\boldsymbol{Y})^k = \sum_{m=0}^{k} \binom{k}{m} \boldsymbol{X}^m \boldsymbol{Y}^{k-m}$. Multiplying the two absolutely convergent series as a Cauchy product,

$$e^{\boldsymbol{X}+\boldsymbol{Y}} = \sum_{k=0}^{\infty} \sum_{m=0}^{k} \frac{\boldsymbol{X}^m \boldsymbol{Y}^{k-m}}{m!(k-m)!} = \left( \sum_{m=0}^{\infty} \frac{\boldsymbol{X}^m}{m!} \right) \left( \sum_{n=0}^{\infty} \frac{\boldsymbol{Y}^n}{n!} \right) = e^{\boldsymbol{X}} e^{\boldsymbol{Y}}$$

$\boldsymbol{X}_1$ commutes with $\boldsymbol{X}_2 + \cdots + \boldsymbol{X}_r$, so applying this repeatedly (induction on $r$):

$$e^{\boldsymbol{X}_1 + \boldsymbol{X}_2 + \cdots + \boldsymbol{X}_r} = e^{\boldsymbol{X}_1} e^{\boldsymbol{X}_2 + \cdots + \boldsymbol{X}_r} = \cdots = e^{\boldsymbol{X}_1} e^{\boldsymbol{X}_2} \cdots e^{\boldsymbol{X}_r}$$

Combining with Part A:

$$\boldsymbol{\Phi}(t,\tau) = e^{\boldsymbol{A}_1 \int_\tau^t f_1(\sigma)\,\mathrm{d}\sigma} e^{\boldsymbol{A}_2 \int_\tau^t f_2(\sigma)\,\mathrm{d}\sigma} \cdots e^{\boldsymbol{A}_r \int_\tau^t f_{r(\sigma)}\,\mathrm{d}\sigma} \quad \blacksquare$$

The factors commute, so their order does not matter. As a numerical check, three commuting matrices (polynomials in one random matrix) were used with $f_j = \cos(t), t, e^{-t}$, and both forms were compared with the numerically integrated STM (Python):

```
commuting A_j (polynomials in a random 3x3 M), f = cos(t), t, e^(-t):
max relative error vs numerical STM:
  exp(sum_j F_j A_j)       7.6e-10
  prod_j exp(F_j A_j)      7.6e-10
```

c) STM for the rotation-like $\boldsymbol{A}(t)$

Split $\boldsymbol{A}(t)$ into constant matrices:

$$\boldsymbol{A}(t) = \cos(3t)\boldsymbol{I} + \sin(3t)\boldsymbol{S}, \quad \boldsymbol{S} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

This has the required form with $r = 2$: $f_1 = \cos(3t)$, $\boldsymbol{A}_1 = \boldsymbol{I}$, $f_2 = \sin(3t)$, and $\boldsymbol{A}_2 = \boldsymbol{S}$. Since $\boldsymbol{I}$ commutes with every matrix, the results above apply, with

$$F_1 = \int_\tau^t \cos(3\sigma)\,\mathrm{d}\sigma = \frac{\sin(3t) - \sin(3\tau)}{3}, \quad F_2 = \int_\tau^t \sin(3\sigma)\,\mathrm{d}\sigma = \frac{\cos(3\tau) - \cos(3t)}{3}$$

By Part B, $\boldsymbol{\Phi} = e^{F_1 \boldsymbol{I}} e^{F_2 \boldsymbol{S}}$. First, $e^{F_1 \boldsymbol{I}} = e^{F_1} \boldsymbol{I}$. Second, since $\boldsymbol{S}^2 = -\boldsymbol{I}$, the series for $e^{\theta \boldsymbol{S}}$ splits into even powers $(-1)^k \boldsymbol{I}$ and odd powers $(-1)^k \boldsymbol{S}$:

$$e^{\theta \boldsymbol{S}} = \cos(\theta)\boldsymbol{I} + \sin(\theta)\boldsymbol{S} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}$$

Therefore

$$\boldsymbol{\Phi}(t,\tau) = e^{(\sin(3t) - \sin(3\tau))/3} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}, \quad \theta(t,\tau) = \frac{\cos(3\tau) - \cos(3t)}{3} \quad \blacksquare$$

Check: $\boldsymbol{\Phi}(\tau,\tau) = \boldsymbol{I}$, and $\frac{\partial \boldsymbol{\Phi}}{\partial t} = \cos(3t)\boldsymbol{\Phi} + \sin(3t)\boldsymbol{S}\boldsymbol{\Phi} = \boldsymbol{A}(t)\boldsymbol{\Phi}$. Python confirms the commutativity and compares the result with the numerically integrated STM:

```
max |A(t1) A(t2) - A(t2) A(t1)| over 20 random pairs = 1.1e-16
max relative error of Phi(t, tau) vs numerical STM = 2.6e-11
```

## Problem 6

For the homogeneous linear system $\dot{\mathbf{x}}(t) = \boldsymbol{A}(t)\mathbf{x}(t)$, find the state transition matrix $\boldsymbol{\Phi}(t,\tau)$ when:

a) $\boldsymbol{A}(t) = \begin{bmatrix} t & 0 \\ 0 & \cos(t) \end{bmatrix}$

b) $\boldsymbol{A}(t) = \begin{bmatrix} \cos(2t) & \cos(2t) \\ 0 & 0 \end{bmatrix}$

c) $\boldsymbol{A}(t) = \begin{bmatrix} -2 & \sin(t) \\ 0 & -1 \end{bmatrix}$

d) $\boldsymbol{A}(t) = \begin{bmatrix} -3 & -1 \\ -1 & -3 \end{bmatrix}$

Show all work for your solutions. In Part D, use a dyadic expansion approach.

a) Diagonal $\boldsymbol{A}(t)$

The equations decouple into two scalar equations, each solved by separation of variables:

$$\dot{x}_1 = t x_1 \quad \Rightarrow \quad x_1(t) = e^{\int_\tau^t \sigma \, \mathrm{d}\sigma} x_1(\tau) = e^{(t^2-\tau^2)/2} x_1(\tau)$$

$$\dot{x}_2 = \cos(t) x_2 \quad \Rightarrow \quad x_2(t) = e^{\int_\tau^t \cos(\sigma) \, \mathrm{d}\sigma} x_2(\tau) = e^{\sin(t) - \sin(\tau)} x_2(\tau)$$

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} e^{(t^2-\tau^2)/2} & 0 \\ 0 & e^{\sin(t)-\sin(\tau)} \end{bmatrix} \quad \blacksquare$$

This agrees with Problem 5: diagonal matrices commute, so $\boldsymbol{\Phi} = e^{\int_\tau^t \boldsymbol{A}(\sigma) \, \mathrm{d}\sigma}$.

b) Rank-one $\boldsymbol{A}(t)$

The second equation is $\dot{x}_2 = 0$, so $x_2(t) = x_2(\tau)$. The first equation is $\dot{x}_1 = \cos(2t)(x_1 + x_2)$. With $w = x_1 + x_2$ and $\dot{w} = \dot{x}_1$:

$$\dot{w} = \cos(2t) w \quad \Rightarrow \quad w(t) = e^{F(t,\tau)} w(\tau), \quad F(t,\tau) = \int_\tau^t \cos(2\sigma) \, \mathrm{d}\sigma = \frac{\sin(2t) - \sin(2\tau)}{2}$$

$$x_1(t) = w(t) - x_2(t) = e^F x_1(\tau) + (e^F - 1) x_2(\tau)$$

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} e^{F(t,\tau)} & e^{F(t,\tau)} - 1 \\ 0 & 1 \end{bmatrix}, \quad F(t,\tau) = \frac{\sin(2t) - \sin(2\tau)}{2} \quad \blacksquare$$

Cross-check with Problem 5: $\boldsymbol{A}(t) = \cos(2t)\boldsymbol{N}$ uses a single constant matrix (so it trivially commutes), and $\boldsymbol{N}$ is idempotent:

$$\boldsymbol{N} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad \boldsymbol{N}^2 = \boldsymbol{N} \quad \Rightarrow \quad e^{F\boldsymbol{N}} = \boldsymbol{I} + \sum_{k=1}^{\infty} \frac{F^k}{k!} \boldsymbol{N} = \boldsymbol{I} + (e^F - 1)\boldsymbol{N}$$

which is the same matrix.

c) Upper-triangular $\boldsymbol{A}(t)$ with time-varying coupling

Here $\boldsymbol{A}(t)$ does not commute with itself at different times:

$$\boldsymbol{A}(t)\boldsymbol{A}(s) = \begin{bmatrix} 4 & -2\sin(s) - \sin(t) \\ 0 & 1 \end{bmatrix} \neq \begin{bmatrix} 4 & -2\sin(t) - \sin(s) \\ 0 & 1 \end{bmatrix} = \boldsymbol{A}(s)\boldsymbol{A}(t) \quad \text{unless} \sin(t) = \sin(s)$$

so the exponential of $\int \boldsymbol{A}$ is *not* the STM. Instead, $\boldsymbol{A}(t)$ is upper triangular, so Problem 2 applies with scalar blocks $A_1 = -2$, $A_2 = -1$, and $A_{1,2} = \sin(t)$:

$$\Phi_1(t,\tau) = e^{-2(t-\tau)}, \quad \Phi_2(t,\tau) = e^{-(t-\tau)}$$

$$\Phi_{1,2}(t,\tau) = \int_\tau^t e^{-2(t-\sigma)} \sin(\sigma) e^{-(\sigma-\tau)} \, \mathrm{d}\sigma = e^{-2t+\tau} \int_\tau^t e^{\sigma} \sin(\sigma) \, \mathrm{d}\sigma$$

Using $\int e^\sigma \sin(\sigma) \, \mathrm{d}\sigma = \frac{1}{2} e^\sigma (\sin(\sigma) - \cos(\sigma))$:

$$\Phi_{1,2}(t,\tau) = \frac{1}{2} e^{-2t+\tau} \left[ e^t (\sin(t) - \cos(t)) - e^\tau (\sin(\tau) - \cos(\tau)) \right] = \frac{1}{2} \left[ e^{-(t-\tau)} (\sin(t) - \cos(t)) - e^{-2(t-\tau)} (\sin(\tau) - \cos(\tau)) \right]$$

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} e^{-2(t-\tau)} & \frac{1}{2}\left[ e^{-(t-\tau)} (\sin(t) - \cos(t)) - e^{-2(t-\tau)} (\sin(\tau) - \cos(\tau)) \right] \\ 0 & e^{-(t-\tau)} \end{bmatrix} \quad \blacksquare$$

Check: $\Phi_{1,2}(\tau,\tau) = 0$, and

$$\frac{\partial \Phi_{1,2}}{\partial t} = e^{-(t-\tau)} \cos(t) + e^{-2(t-\tau)} (\sin(\tau) - \cos(\tau)) = -2\Phi_{1,2} + \sin(t) e^{-(t-\tau)}$$

which is the $(1,2)$ entry of $\boldsymbol{A}(t)\boldsymbol{\Phi}$.

d) Dyadic expansion for constant $\boldsymbol{A}$

For a constant, diagonalizable $\boldsymbol{A} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{-1}$, let $\mathbf{v}_i$ be the eigenvectors (columns of $\boldsymbol{V}$) and $\mathbf{w}_i^\mathsf{T}$ the reciprocal basis (rows of $\boldsymbol{V}^{-1}$, so $\mathbf{w}_i^\mathsf{T}\mathbf{v}_j = \delta_{ij}$). Then $\boldsymbol{A}$ has the dyadic expansion

$$\boldsymbol{A} = \sum_i \lambda_i \mathbf{v}_i \mathbf{w}_i^\mathsf{T} \quad \Rightarrow \quad \boldsymbol{A}^k = \sum_i \lambda_i^k \mathbf{v}_i \mathbf{w}_i^\mathsf{T} \quad \Rightarrow \quad \boldsymbol{\Phi}(t,\tau) = e^{\boldsymbol{A}(t-\tau)} = \sum_i e^{\lambda_i (t-\tau)} \mathbf{v}_i \mathbf{w}_i^\mathsf{T}$$

The eigenvalues satisfy $\det(\lambda \boldsymbol{I} - \boldsymbol{A}) = (\lambda+3)^2 - 1 = (\lambda+2)(\lambda+4) = 0$, so $\lambda_1 = -2$ and $\lambda_2 = -4$. Eigenvectors:

$$(\boldsymbol{A} + 2\boldsymbol{I})\mathbf{v}_1 = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix} \mathbf{v}_1 = \mathbf{0} \Rightarrow \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad (\boldsymbol{A} + 4\boldsymbol{I})\mathbf{v}_2 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \mathbf{v}_2 = \mathbf{0} \Rightarrow \mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Reciprocal basis:

$$\boldsymbol{V}^{-1} = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}^{-1} = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \quad \Rightarrow \quad \mathbf{w}_1^\mathsf{T} = \frac{1}{2}[1, -1], \quad \mathbf{w}_2^\mathsf{T} = \frac{1}{2}[1, 1]$$

The dyads are

$$\mathbf{v}_1 \mathbf{w}_1^\mathsf{T} = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad \mathbf{v}_2 \mathbf{w}_2^\mathsf{T} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

They sum to $\boldsymbol{I}$, and $-2\mathbf{v}_1\mathbf{w}_1^\mathsf{T} - 4\mathbf{v}_2\mathbf{w}_2^\mathsf{T} = \boldsymbol{A}$, as expected. Therefore

$$\boldsymbol{\Phi}(t,\tau) = \frac{e^{-2(t-\tau)}}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \frac{e^{-4(t-\tau)}}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} e^{-2(t-\tau)} + e^{-4(t-\tau)} & -e^{-2(t-\tau)} + e^{-4(t-\tau)} \\ -e^{-2(t-\tau)} + e^{-4(t-\tau)} & e^{-2(t-\tau)} + e^{-4(t-\tau)} \end{bmatrix} \quad \blacksquare$$

Each STM above was compared with the numerically integrated STM. For Part C, Python also shows that $\exp(\int \boldsymbol{A})$ is not the STM, and for Part D it compares the dyadic sum with a matrix exponential:

```
max relative error of Phi(t, tau) vs numerical STM:
  a) 5.0e-11
  b) 3.2e-11
  c) 4.1e-11
  d) 2.1e-11
c) |A(1) A(2) - A(2) A(1)| = 0.068
c) max |exp(int A) - Phi| = 0.021  (exp(int A) is not the STM)
d) eigenvalues = [-4. -2.]
d) max relative |dyadic sum - e^(At)| = 1.5e-12
```

## Problem 7

Consider a time-periodic system with

$$\boldsymbol{A}(t) = \begin{bmatrix} -1 + 1.5\cos^2(t) & 1 - 1.5\sin(t)\cos(t) \\ -1 - 1.5\sin(t)\cos(t) & -1 + 1.5\sin^2(t) \end{bmatrix}$$

a) Find the eigenvalues of $\boldsymbol{A}(t)$, show that they do not change with time, and are located in the left-half of the complex plane.

b) Verify that the two fundamental solutions for this system are:

$$\boldsymbol{\psi}_1(t) = \begin{bmatrix} e^{-t}\sin(t) \\ e^{-t}\cos(t) \end{bmatrix} \quad \boldsymbol{\psi}_2(t) = \begin{bmatrix} e^{\frac{t}{2}}\cos(t) \\ -e^{\frac{t}{2}}\sin(t) \end{bmatrix}$$

c) Determine the state transition matrix for this system. Is this system stable; i.e. does $\mathbf{x}(t) \to \mathbf{0}$ from every initial condition $\mathbf{x}_0$?

d) Comparing your results in Part A and Part C, comment on the utility of the eigenvalues of a time-varying $\boldsymbol{A}(t)$ for analyzing stability.

a) Frozen-time eigenvalues

Abbreviate $s = \sin(t)$ and $c = \cos(t)$. The trace and determinant of $\boldsymbol{A}(t)$ are

$$\operatorname{trace}\boldsymbol{A}(t) = -2 + 1.5(c^2 + s^2) = -\frac{1}{2}$$

$$\begin{aligned} \det\boldsymbol{A}(t) &= (-1 + 1.5c^2)(-1 + 1.5s^2) - (1 - 1.5sc)(-1 - 1.5sc) \\ &= (1 - 1.5(c^2 + s^2) + 2.25s^2c^2) + (1 - 2.25s^2c^2) = \frac{1}{2} \end{aligned}$$

Both are independent of $t$, so the characteristic polynomial $\lambda^2 - \operatorname{trace}(\boldsymbol{A})\lambda + \det(\boldsymbol{A}) = \lambda^2 + \frac{1}{2}\lambda + \frac{1}{2}$ is the same at every instant:

$$\lambda_{1,2} = \frac{-\frac{1}{2} \pm \sqrt{\frac{1}{4} - 2}}{2} = -\frac{1}{4} \pm j\frac{\sqrt{7}}{4} \quad \blacksquare$$

The eigenvalues do not change with time, and $\operatorname{Re}(\lambda_{1,2}) = -\frac{1}{4} < 0$, so they always lie in the open left half-plane. Python confirms this on a dense grid over a period:

```
1  eigenvalues of A(t) on a 721-point grid over [0, 2 pi]:
2    real part in [-0.250000, -0.250000]
3    |imaginary part| in [0.661438, 0.661438],  sqrt(7)/4 = 0.661438
```

b) Fundamental solutions

Differentiating $\boldsymbol{\psi}_1$ and computing $\boldsymbol{A}\boldsymbol{\psi}_1$ (again with $s = \sin(t)$, $c = \cos(t)$):

$$\dot{\boldsymbol{\psi}}_1 = e^{-t}\begin{bmatrix} c - s \\ -s - c \end{bmatrix}, \quad \boldsymbol{A}\boldsymbol{\psi}_1 = e^{-t}\begin{bmatrix} (-1 + 1.5c^2)s + (1 - 1.5sc)c \\ (-1 - 1.5sc)s + (-1 + 1.5s^2)c \end{bmatrix} = e^{-t}\begin{bmatrix} c - s \\ -s - c \end{bmatrix}$$

Similarly for $\boldsymbol{\psi}_2$, using $c^2 + s^2 = 1$:

$$\dot{\boldsymbol{\psi}}_2 = e^{t/2}\begin{bmatrix} \frac{1}{2}c - s \\ -\frac{1}{2}s - c \end{bmatrix}, \quad \boldsymbol{A}\boldsymbol{\psi}_2 = e^{t/2}\begin{bmatrix} (-1 + 1.5c^2)c - (1 - 1.5sc)s \\ (-1 - 1.5sc)c - (-1 + 1.5s^2)s \end{bmatrix} = e^{t/2}\begin{bmatrix} -c + 1.5c - s \\ -c + s - 1.5s \end{bmatrix}$$

which equals $\dot{\boldsymbol{\psi}}_2$. Both are therefore solutions of $\dot{\mathbf{x}} = \boldsymbol{A}(t)\mathbf{x}$. They are linearly independent for every $t$:

$$\det[\boldsymbol{\psi}_1 \;\; \boldsymbol{\psi}_2] = e^{-t}e^{t/2}\det\begin{bmatrix} s & c \\ c & -s \end{bmatrix} = -e^{-t/2} \neq 0 \quad \blacksquare$$

This is consistent with the Abel–Jacobi–Liouville formula, $\det \boldsymbol{\Psi}(t) = \det \boldsymbol{\Psi}(0) e^{\int_0^t \operatorname{trace} \boldsymbol{A}} = -e^{-t/2}$. Hence $\boldsymbol{\psi}_1$ and $\boldsymbol{\psi}_2$ form a fundamental set. Python confirms both properties:

```
max relative |dPsi/dt - A(t) Psi(t)| on [-3, 6] = 1.8e-15
max |det Psi(t) + e^(-t/2)| on [-3, 6] = 1.8e-15
```

c) State transition matrix and stability

Collect the fundamental solutions into the fundamental matrix $\boldsymbol{\Psi}(t)$, whose columns are $\boldsymbol{\psi}_1(t)$ and $\boldsymbol{\psi}_2(t)$. With $\det \boldsymbol{\Psi}(\tau) = -e^{-\tau/2}$:

$$\boldsymbol{\Psi}(t) = \begin{bmatrix} e^{-t}\sin(t) & e^{t/2}\cos(t) \\ e^{-t}\cos(t) & -e^{t/2}\sin(t) \end{bmatrix}, \quad \boldsymbol{\Psi}^{-1}(\tau) = \begin{bmatrix} e^{\tau}\sin(\tau) & e^{\tau}\cos(\tau) \\ e^{-\tau/2}\cos(\tau) & -e^{-\tau/2}\sin(\tau) \end{bmatrix}$$

Then $\boldsymbol{\Phi}(t,\tau) = \boldsymbol{\Psi}(t)\boldsymbol{\Psi}^{-1}(\tau)$:

$$\boldsymbol{\Phi}(t,\tau) = \begin{bmatrix} e^{-(t-\tau)}\sin(t)\sin(\tau) + e^{(t-\tau)/2}\cos(t)\cos(\tau) & e^{-(t-\tau)}\sin(t)\cos(\tau) - e^{(t-\tau)/2}\cos(t)\sin(\tau) \\ e^{-(t-\tau)}\cos(t)\sin(\tau) - e^{(t-\tau)/2}\sin(t)\cos(\tau) & e^{-(t-\tau)}\cos(t)\cos(\tau) + e^{(t-\tau)/2}\sin(t)\sin(\tau) \end{bmatrix} \quad \blacksquare$$

Clearly $\boldsymbol{\Phi}(\tau,\tau) = \boldsymbol{I}$. Multiplying out confirms the equivalent factored form

$$\boldsymbol{\Phi}(t,\tau) = \boldsymbol{R}(t)\begin{bmatrix} e^{(t-\tau)/2} & 0 \\ 0 & e^{-(t-\tau)} \end{bmatrix}\boldsymbol{R}(\tau)^{\mathsf{T}}, \quad \boldsymbol{R}(t) = \begin{bmatrix} \cos(t) & \sin(t) \\ -\sin(t) & \cos(t) \end{bmatrix}$$

where $\boldsymbol{R}(t)$ is a rotation matrix.

*Stability.* The system is **not** stable. Starting from $\mathbf{x}_0 = [1,0]^{\mathsf{T}}$ at $t_0 = 0$ gives

$$\mathbf{x}(t) = \boldsymbol{\Phi}(t,0)\begin{bmatrix} 1 \\ 0 \end{bmatrix} = \boldsymbol{\psi}_2(t) = e^{t/2}\begin{bmatrix} \cos(t) \\ -\sin(t) \end{bmatrix}, \quad \|\mathbf{x}(t)\| = e^{t/2} \to \infty \quad \blacksquare$$

In general, $\mathbf{x}(t) = \boldsymbol{\Psi}(t)\boldsymbol{\Psi}^{-1}(t_0)\mathbf{x}_0$ decays only if the $\boldsymbol{\psi}_2$ coefficient $e^{-t_0/2}\big(\cos(t_0)x_{0,1} - \sin(t_0)x_{0,2}\big)$ is zero. That is, $\mathbf{x}_0$ must be parallel to $\boldsymbol{\psi}_1(t_0)$, a one-dimensional set. Every other initial state diverges like $e^{t/2}$.

The same conclusion follows from Floquet theory: $\boldsymbol{A}(t)$ has period $\pi$, and the monodromy matrix is $\boldsymbol{\Phi}(\pi,0) = \operatorname{diag}\big(-e^{\pi/2}, -e^{-\pi}\big)$. One Floquet multiplier has magnitude $e^{\pi/2} \approx 4.81 > 1$.

```
max relative error of Phi(t, tau) vs numerical STM = 4.6e-11
monodromy matrix Phi(pi, 0) =
[[-4.8105  0.    ]
 [ 0.     -0.0432]]
Floquet multipliers = [-4.8105 -0.0432]
-e^(pi/2) = -4.8105, -e^(-pi) = -0.0432
```

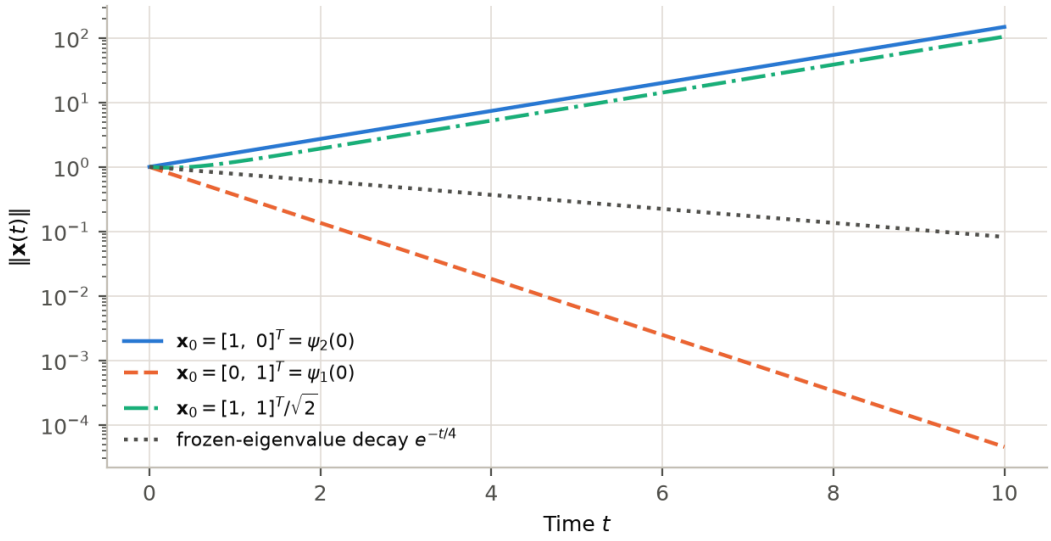



Figure 3: State norm for three initial states, compared with the $e^{-t/4}$ decay suggested by the frozen-time eigenvalues.

d) Eigenvalues of a time-varying $\boldsymbol{A}(t)$

In Part A, the eigenvalues of $\boldsymbol{A}(t)$ are constant and in the left half-plane for every $t$. Read as if the system were LTI, they would predict decay like $e^{-t/4}$. Yet Part C shows that almost every solution grows like $e^{t/2}$. For a time-varying system, the instantaneous ("frozen-time") eigenvalues of $\boldsymbol{A}(t)$ do not determine stability:

- Left-half-plane frozen eigenvalues are not sufficient for stability, as this example shows. Neither are they necessary: frozen eigenvalues in the right half-plane can coexist with asymptotic stability (an example follows below).
- The reason is visible here. $\boldsymbol{A}(t) = \boldsymbol{R}(t)\boldsymbol{A}(0)\boldsymbol{R}(t)^\mathsf{T}$ is a rotating copy of $\boldsymbol{A}(0)$, with $\boldsymbol{R}(t)$ from Part C. A similarity transformation preserves the frozen eigenvalues, but it ignores the motion of the coordinate frame. In the rotating coordinates $\mathbf{z} = \boldsymbol{R}(t)^\mathsf{T}\mathbf{x}$, using $\dot{\boldsymbol{R}} = \boldsymbol{S}\boldsymbol{R}$, the dynamics become time-invariant:

$$\dot{\mathbf{z}} = (\boldsymbol{A}(0) - \boldsymbol{S})\mathbf{z}, \quad \boldsymbol{A}(0) = \begin{bmatrix} \frac{1}{2} & 1 \\ -1 & -1 \end{bmatrix}, \quad \boldsymbol{S} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad \boldsymbol{A}(0) - \boldsymbol{S} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & -1 \end{bmatrix}$$

  This has an unstable eigenvalue $+\frac{1}{2}$. The frame rotation contributes $-\boldsymbol{S}$, which the frozen eigenvalues never see. The same mechanism can also stabilize a system. Rotating $\boldsymbol{A}_0 = \begin{bmatrix} -1 & 1 \\ 9 & -1 \end{bmatrix}$ instead (frozen eigenvalues 2 and $-4$) gives $\boldsymbol{A}_0 - \boldsymbol{S} = \begin{bmatrix} -1 & 0 \\ 10 & -1 \end{bmatrix}$, which is Hurwitz. That system is asymptotically stable even though one frozen eigenvalue is in the right half-plane for every $t$.
- Stability of an LTV system must be judged from $\boldsymbol{\Phi}(t, \tau)$ itself (boundedness and decay), from Lyapunov arguments, or, for periodic systems, from the Floquet multipliers (the eigenvalues of the monodromy matrix $\boldsymbol{\Phi}(T, 0)$). Frozen-time eigenvalues are only trustworthy under extra conditions. One case is when $\boldsymbol{A}(t)$ is bounded, commutes with itself at all times (as in Problem 5), *and* its eigenvalues satisfy $\mathrm{Re}(\lambda_i(t)) \leq -\varepsilon < 0$ uniformly. Another is when $\boldsymbol{A}(t)$ is bounded, satisfies the same uniform bound $\mathrm{Re}(\lambda_i(t)) \leq -\varepsilon$, and varies slowly enough compared with that decay rate. Commutativity alone is not enough: $\dot{x} = -x/(1+t)^2$ has a negative frozen eigenvalue for all $t$, yet $x(t) = x_0 e^{1/(1+t)-1} \to x_0/e \neq 0$.

So the eigenvalues of a time-varying $\boldsymbol{A}(t)$ are not a reliable stability test. ■

## Code

./src/index.py Python

```python
from functools import reduce
from pathlib import Path

import control as ct
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import quad, quad_vec, solve_ivp
from scipy.linalg import expm

matplotlib.use("Agg")

ROOT = Path(__file__).resolve().parents[1]
FIGURES = ROOT / "outputs/figures"
TEXT = ROOT / "outputs/text"

# series colours (first three categorical slots, validated as a set) and chart ink
BLUE, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
INK, MUTED = "#0b0b0b", "#52514e"
plt.rcParams.update(
    {
        "axes.edgecolor": "#c3c2b7",
        "axes.labelcolor": INK,
        "axes.spines.right": False,
        "axes.spines.top": False,
        "axes.titlesize": 9,
        "font.size": 9,
        "grid.color": "#e1e0d9",
        "grid.linewidth": 0.6,
        "legend.frameon": False,
        "lines.linewidth": 1.6,
        "text.color": INK,
        "xtick.color": MUTED,
        "ytick.color": MUTED,
    }
)


def heading(problem):
    print(f"\n------\n {problem}\n------")


def report(name, content):
    print(content)
```

```
45      (TEXT / f"{name}.txt").write_text(content + "\n", encoding="utf-8")
46
47
48  def save(fig, name):
49      fig.tight_layout()
50      fig.savefig(FIGURES / f"{name}.png", dpi=200)
51      plt.close(fig)
52
53
54  def show(M, precision=4):
55      """Format an array, rounding away floating-point dust (and negative zeros)."""
56      return np.array2string(
57          np.round(np.asarray(M), 10) + 0.0, precision=precision, suppress_small=True
58      )
59
60
61  def poly_str(coeffs, var="s"):
62      """Format polynomial coefficients (highest power first) as text."""
63      coeffs = np.trim_zeros(np.round(np.real(coeffs), 6) + 0.0, "f")
64      terms = []
65      for power, c in zip(range(coeffs.size - 1, -1, -1), coeffs):
66          if c == 0:
67              continue
68          mag = "" if abs(c) == 1 and power > 0 else f"{abs(c):g}"
69          sym = "" if power == 0 else var if power == 1 else f"{var}^{power}"
70          terms.append(("-" if c < 0 else "+", f"{mag} {sym}".strip()))
71      if not terms:
72          return "0"
73      text = ("-" if terms[0][0] == "-" else "") + terms[0][1]
74      return text + "".join(f" {sign} {body}" for sign, body in terms[1:])
75
76
77  def tf_str(g):
78      """Format a SISO transfer function with a monic denominator."""
79      num, den = np.asarray(g.num[0][0]), np.asarray(g.den[0][0])
80      num, den = num / den[0], den / den[0]
81      numerator = poly_str(num)
82      if numerator == "0":
83          return "0"
84      if any(op in numerator[1:] for op in "+-"):
85          numerator = f"({numerator})"
86      return f"{numerator} / ({poly_str(den)})"
87
88
89  def channel_tfs(A, B, C, cancel=True):
90      """Transfer function of every input-output channel of the model (A, B, C, 0)."""
```

```
91      G = {}
92      for i in range(C.shape[0]):
93          for j in range(B.shape[1]):
94              g = ct.tf(ct.ss(A, B[:, [j]], C[[i], :], 0))
95              num = np.array(g.num[0][0], dtype=float)
96              num[np.abs(num) < 1e-9] = 0.0  # round-off in an identically zero channel
97              g = ct.tf(num, g.den[0][0]) if num.any() else ct.tf([0.0], [1.0])
98              G[i, j] = g.minreal(tol=1e-6) if cancel else g
99      return G
100
101
102 def stm_numeric(A, tau, t):
103     """Integrate dPhi/dt = A(t) Phi from Phi(tau, tau) = I, returning Phi(t_k, tau)."""
104     n = A(tau).shape[0]
105     sol = solve_ivp(
106         lambda s, phi: (A(s) @ phi.reshape(n, n)).ravel(),
107         (tau, t[-1]),
108         np.eye(n).ravel(),
109         method="DOP853",
110         t_eval=t,
111         rtol=1e-11,
112         atol=1e-12,
113     )
114     assert sol.success
115     return sol.y.T.reshape(-1, n, n)
116
117
118 def stm_error(A, Phi, taus=(-1.0, 0.0, 1.5), span=3.0):
119     """Largest relative mismatch between a closed-form STM and the numerical STM."""
120     worst = 0.0
121     for tau in taus:
122         for end in (tau + span, tau - span):
123             t = np.linspace(tau, end, 31)
124             numeric = stm_numeric(A, tau, t)
125             exact = np.array([Phi(tk, tau) for tk in t])
126             error = np.abs(numeric - exact) / np.maximum(1.0, np.abs(exact))
127             worst = max(worst, error.max())
128     assert worst < 1e-7
129     return worst
130
131
132 def main():
133     FIGURES.mkdir(parents=True, exist_ok=True)
134     TEXT.mkdir(parents=True, exist_ok=True)
135     rng = np.random.default_rng(641)
136
```

```
    #########
    # p01 #
    #########
    heading("p01")
    A = np.array([[0, 0, -13], [1, 0, -17], [0, 1, -5]], dtype=float)
    B = np.array([[13, 1], [4, 1], [1, 0]], dtype=float)
    C = np.array([[1, -1, 1], [0, 1, -1]], dtype=float)

    # a) eigenvectors from the companion structure, v(lambda) = [l^2 + 5l + 17, l + 5, 1]
    charpoly = np.poly(A)
    assert np.allclose(charpoly, [1, 5, 17, 13])

    def eigenvector(lam):
        return np.array([lam**2 + 5 * lam + 17, lam + 5, 1])

    lam_c, lam_r = -2 + 3j, -1.0
    for lam in (lam_c, lam_r):
        assert np.allclose(A @ eigenvector(lam), lam * eigenvector(lam))
    v = eigenvector(lam_c)
    P = np.column_stack([v.real, v.imag, eigenvector(lam_r)])
    P_inv = np.linalg.inv(P)
    Lam = P_inv @ A @ P
    B_bar, C_bar = P_inv @ B, C @ P
    assert np.allclose(Lam, [[-2, 3, 0], [-3, -2, 0], [0, 0, -1]])
    report(
        "s01a",
        f"characteristic polynomial: {poly_str(charpoly)}\n"
        f"eigenvalues = {show(np.sort_complex(np.linalg.eigvals(A)))}\n"
        f"v(-2 + 3j) = {show(v)}\n"
        f"v(-1) = {show(eigenvector(lam_r))}\n\n"
        f"P =\n{show(P)}\n\n30 P^-1 =\n{show(30 * P_inv)}\n\n"
        f"Lambda = P^-1 A P =\n{show(Lam)}\n\n"
        f"B_bar = P^-1 B =\n{show(B_bar)}\n\nC_bar = C P =\n{show(C_bar)}",
    )

    # b) homogeneous response from a single-mode initial state
    t = np.linspace(0, 3, 601)
    p1, p2, p3 = P.T
    x_osc = np.array([expm(A * tk) @ p1 for tk in t])
    x_exp = np.array([expm(A * tk) @ p3 for tk in t])
    z_osc, z_exp = x_osc @ P_inv.T, x_exp @ P_inv.T
    osc_hand = np.outer(np.exp(-2 * t) * np.cos(3 * t), p1) - np.outer(
        np.exp(-2 * t) * np.sin(3 * t), p2
    )
    exp_hand = np.outer(np.exp(-t), p3)
    report(
```

```
        "s01b",
        f"x0 = p1: max |z_3(t)| = {np.abs(z_osc[:, 2]).max():.1e}, "
        f"max |x(t) - e^(-2t)(cos(3t) p1 - sin(3t) p2)| = "
        f"{np.abs(x_osc - osc_hand).max():.1e}\n"
        f"x0 = p3: max |z_1(t)|, |z_2(t)| = {np.abs(z_exp[:, :2]).max():.1e}, "
        f"max |x(t) - e^(-t) p3| = {np.abs(x_exp - exp_hand).max():.1e}",
    )
    assert np.abs(z_osc[:, 2]).max() < 1e-9 and np.abs(z_exp[:, :2]).max() < 1e-9
    assert np.allclose(x_osc, osc_hand) and np.allclose(x_exp, exp_hand)

    fig, axes = plt.subplots(2, 2, figsize=(7.2, 4.6), sharex=True)
    styles = [(BLUE, "-"), (ORANGE, "--"), (AQUA, "-.")]
    cases = [
        (r"$\mathbf{x}_0 = \mathbf{p}_1$ (oscillatory mode)", x_osc, z_osc),
        (r"$\mathbf{x}_0 = \mathbf{p}_3$ (exponential mode)", x_exp, z_exp),
    ]
    for col, (title, x, z) in enumerate(cases):
        for k, (color, ls) in enumerate(styles):
            axes[0, col].plot(t, x[:, k], color=color, ls=ls, label=f"$x_{k + 1}$")
            axes[1, col].plot(t, z[:, k], color=color, ls=ls, label=f"$z_{k + 1}$")
        axes[0, col].set_title(title)
        axes[1, col].set_xlabel("Time")
    axes[0, 0].set_ylabel(r"State $\mathbf{x}(t)$")
    axes[1, 0].set_ylabel(r"Modal coordinates $\mathbf{P}^{-1}\mathbf{x}(t)$")
    for ax in axes.flat:
        ax.grid(True)
        ax.legend(fontsize=8, ncol=3, loc="upper right")
    save(fig, "s01b")

    # c) transfer function matrix from both realizations
    G_orig = channel_tfs(A, B, C)
    G_modal = channel_tfs(Lam, B_bar, C_bar)
    G_full = channel_tfs(A, B, C, cancel=False)
    for key in G_orig:
        assert tf_str(G_orig[key]) == tf_str(G_modal[key])

    def G_hand(s):
        return np.array(
            [[10 / (s + 1), 0], [3 / (s + 1), (s + 4) / (s**2 + 4 * s + 13)]]
        )

    def G_eval(Am, Bm, Cm, s):
        return Cm @ np.linalg.solve(s * np.eye(3) - Am, Bm)

    s_grid = 1j * np.logspace(-2, 2, 200)
    diff_models = max(
```

```
229         np.abs(G_eval(A, B, C, s) - G_eval(Lam, B_bar, C_bar, s)).max() for s in s_grid
230     )
231     diff_hand = max(np.abs(G_eval(A, B, C, s) - G_hand(s)).max() for s in s_grid)
232     assert diff_models < 1e-9 and diff_hand < 1e-9
233     lines = ["original model, C (sI - A)^-1 B before cancellation:"]
234     lines += [f"  G{i + 1}{j + 1}(s) = {tf_str(g)}" for (i, j), g in G_full.items()]
235     for label, G in (("original", G_orig), ("modal", G_modal)):
236         lines.append(f"{label} model after pole-zero cancellation:")
237         lines += [f"  G{i + 1}{j + 1}(s) = {tf_str(g)}" for (i, j), g in G.items()]
238     lines.append(f"max |G_original(jw) - G_modal(jw)| = {diff_models:.1e}")
239     lines.append(f"max |G_original(jw) - G_hand(jw)| = {diff_hand:.1e}")
240     report("s01c", "\n".join(lines))
241
242     #########
243     # p02 #
244     #########
245     heading("p02")
246
247     # sample upper block-triangular system: A_1 is 2x2, A_2 is 1x1, A_12 is 2x1
248     def A_1(t):
249         return np.array([[-1.0, 1 + np.sin(t)], [-0.5, -2 + 0.5 * np.cos(t)]])
250
251     def A_2(t):
252         return np.array([[-0.5 + 0.3 * np.cos(t)]])
253
254     def A_12(t):
255         return np.array([[1 + 0.5 * t], [np.cos(t)]])
256
257     def A_full(t):
258         return np.block([[A_1(t), A_12(t)], [np.zeros((1, 2)), A_2(t)]])
259
260     def fundamental(A_t, n, end):
261         sol = solve_ivp(
262             lambda s, x: (A_t(s) @ x.reshape(n, n)).ravel(),
263             (0, end),
264             np.eye(n).ravel(),
265             method="DOP853",
266             dense_output=True,
267             rtol=1e-12,
268             atol=1e-13,
269         )
270         return lambda s: sol.sol(s).reshape(n, n)
271
272     X_1, X_2 = fundamental(A_1, 2, 4.0), fundamental(A_2, 1, 4.0)
273
274     def Phi_1(t, tau):
```

```
275         return X_1(t) @ np.linalg.inv(X_1(tau))
276
277     def Phi_2(t, tau):
278         return X_2(t) @ np.linalg.inv(X_2(tau))
279
280     def Phi_12(t, tau):
281         def integrand(s):
282             return (Phi_1(t, s) @ A_12(s) @ Phi_2(s, tau)).ravel()
283
284         return quad_vec(integrand, tau, t, epsabs=1e-12, epsrel=1e-11)[0].reshape(2, 1)
285
286     tau = 0.5
287     t = np.linspace(tau, 3.5, 31)
288     Phi = stm_numeric(A_full, tau, t)
289     errors = {
290         "Phi_21": max(np.abs(Ph[2:, :2]).max() for Ph in Phi),
291         "Phi_11 - Phi_1": max(
292             np.abs(Ph[:2, :2] - Phi_1(tk, tau)).max() for tk, Ph in zip(t, Phi)
293         ),
294         "Phi_22 - Phi_2": max(
295             np.abs(Ph[2:, 2:] - Phi_2(tk, tau)).max() for tk, Ph in zip(t, Phi)
296         ),
297         "Phi_12 - integral formula": max(
298             np.abs(Ph[:2, 2:] - Phi_12(tk, tau)).max() for tk, Ph in zip(t, Phi)
299         ),
300     }
301     assert max(errors.values()) < 1e-8
302     report(
303         "s02",
304         "sample system: A_1(t) is 2x2, A_12(t) is 2x1, A_2(t) is 1x1, "
305         f"tau = {tau}, t in [{tau}, {t[-1]}]\n"
306         + "\n".join(f"max |{k}| = {e:.1e}" for k, e in errors.items()),
307     )
308
309     #########
310     # p03 #
311     #########
312     heading("p03")
313
314     def A3(t):
315         return np.array([[0.0, t], [0.0, -1.0]])
316
317     def B3(t):
318         return np.array([0.0, np.exp(-t)])
319
320     def Phi3(t, tau):
```

```
321         decay = np.exp(-(t - tau))
322         return np.array([[1.0, (tau + 1) - (t + 1) * decay], [0.0, decay]])
323
324     def G3(t, tau):
325         return t * ((tau + 1) * np.exp(-tau) - (t + 1) * np.exp(-t)) * (t >= tau)
326
327     def y3(t):
328         return t * (3 / np.e - 3 + (2 * np.e * (t + 1) - t**2 - t - 1) * np.exp(-t))
329
330     # a) closed-form STM against the numerical STM
331     report(
332         "s03a",
333         "max relative error of Phi(t, tau) vs numerical STM = "
334         f"{stm_error(A3, Phi3):.1e}",
335     )
336
337     # b) an impulse at tau sets x(tau+) = B(tau); the output that follows is G(t, tau)
338     worst = 0.0
339     for tau in (0.0, 1.0, 2.5):
340         t = np.linspace(tau, tau + 5, 51)
341         sol = solve_ivp(
342             lambda s, x: A3(s) @ x,
343             (tau, t[-1]),
344             B3(tau),
345             method="DOP853",
346             t_eval=t,
347             rtol=1e-11,
348             atol=1e-13,
349         )
350         worst = max(worst, np.abs(t * sol.y[0] - G3(t, tau)).max())
351     assert worst < 1e-9
352     report(
353         "s03b",
354         f"max |simulated impulse response - G(t, tau)| for tau = 0, 1, 2.5: {worst:.1e}",
355     )
356
357     # c) forced response from x(1) = [1, -2] with u = 1
358     x_1 = np.array([1.0, -2.0])
359     t = np.linspace(1, 10, 451)
360     sol = solve_ivp(
361         lambda s, x: A3(s) @ x + B3(s),
362         (1, 10),
363         x_1,
364         method="DOP853",
365         t_eval=t,
366         rtol=1e-11,
```

```
367         atol=1e-12,
368     )
369     y_sim = t * sol.y[0]
370
371     def superposition(t):
372         zero_input = t * (Phi3(t, 1.0) @ x_1)[0]
373         zero_state = quad(lambda s: G3(t, s), 1, t)[0]
374         return zero_input + zero_state
375
376     y_superposition = np.array([superposition(tk) for tk in t])
377     assert np.allclose(y3(t), y_sim, atol=1e-8)
378     assert np.allclose(y3(t), y_superposition, atol=1e-8)
379     report(
380         "s03c",
381         f"y(1) = {y3(1.0):.6f}\n"
382         f"max |y(t) - simulated y(t)| on [1, 10] = {np.abs(y3(t) - y_sim).max():.1e}\n"
383         "max |y(t) - (C Phi(t, 1) x(1) + integral of G(t, s) ds)| = "
384         f"{np.abs(y3(t) - y_superposition).max():.1e}\n"
385         f"y(t) / t -> 3/e - 3 = {3 / np.e - 3:.6f}",
386     )
387
388     y_zi = t * (-3 + 2 * (t + 1) * np.exp(1 - t))
389     y_zs = t * (3 / np.e - (t**2 + t + 1) * np.exp(-t))
390     fig, ax = plt.subplots(figsize=(6.4, 3.4))
391     ax.plot(t, y3(t), color=BLUE, label=r"$y(t)$, closed form")
392     ax.plot(
393         t[::15],
394         y_sim[::15],
395         "o",
396         color=BLUE,
397         markerfacecolor="white",
398         markersize=5,
399         label="numerical simulation",
400     )
401     ax.plot(t, y_zi, color=ORANGE, ls="--", label="zero-input part")
402     ax.plot(t, y_zs, color=AQUA, ls="-.", label="zero-state part")
403     ax.plot(t, (3 / np.e - 3) * t, color=MUTED, ls=":", label=r"asymptote $(3/e - 3)t$")
404     ax.set(xlabel=r"Time $t$", ylabel=r"Output $y(t)$")
405     ax.grid(True)
406     ax.legend(fontsize=8)
407     save(fig, "s03c")
408
409     #########
410     # p04 #
411     #########
412     heading("p04")
```

```
413     J = np.zeros((5, 5))
414     J[:2, :2] = [[-1, 3], [-3, -1]]
415     J[2:, 2:] = [[-3, 1, 0], [0, -3, 1], [0, 0, -3]]
416
417     def eJt(t):
418         c, s = np.cos(3 * t), np.sin(3 * t)
419         E = np.zeros((5, 5))
420         E[:2, :2] = np.exp(-t) * np.array([[c, s], [-s, c]])
421         E[2:, 2:] = np.exp(-3 * t) * np.array([[1, t, t**2 / 2], [0, 1, t], [0, 0, 1]])
422         return E
423
424     # any invertible P works; build an A with the stated structure and check it
425     P4 = rng.normal(size=(5, 5))
426     P4_inv = np.linalg.inv(P4)
427     A4 = P4 @ J @ P4_inv
428     r = np.polymul([1, 2, 10], np.poly([-3, -3, -3]))
429     ranks = [
430         int(
431             np.linalg.matrix_rank(
432                 np.linalg.matrix_power(A4 + 3 * np.eye(5), k), rtol=1e-8
433             )
434         )
435         for k in (1, 2, 3)
436     ]
437     assert np.allclose(np.poly(A4), r, atol=1e-6) and ranks == [4, 3, 2]
438     t = np.linspace(0, 4, 81)
439     err_a = max(np.abs(P4_inv @ expm(A4 * tk) @ P4 - eJt(tk)).max() for tk in t)
440
441     def y_b(t):
442         return np.array([0, 0, t * np.exp(-3 * t), np.exp(-3 * t), 0])
443
444     def y_c(t):
445         return np.array(
446             [np.exp(-t) * np.sin(3 * t), np.exp(-t) * np.cos(3 * t), 0, 0, 0]
447         )
448
449     err_b = max(np.abs(P4_inv @ expm(A4 * tk) @ P4[:, 3] - y_b(tk)).max() for tk in t)
450     err_c = max(np.abs(P4_inv @ expm(A4 * tk) @ P4[:, 1] - y_c(tk)).max() for tk in t)
451     assert max(err_a, err_b, err_c) < 1e-9
452     report(
453         "s04",
454         f"r(lambda) = {poly_str(r, 'l')}\n"
455         f"charpoly(A) = {poly_str(np.poly(A4), 'l')}  (A = P J P^-1, random P)\n"
456         f"rank (A + 3I)^k for k = 1, 2, 3: {ranks}\n"
457         f"a) max |P^-1 e^(At) P - closed form| = {err_a:.1e}\n"
458         f"b) max |P^-1 e^(At) p_4 - closed form| = {err_b:.1e}\n"
```

```
459         f"c) max |P^-1 e^(At) p_2 - closed form| = {err_c:.1e}",
460     )
461
462     #########
463     # p05 #
464     #########
465     heading("p05")
466
467     # a), b) commuting family: polynomials in one random matrix M
468     M = 0.5 * rng.normal(size=(3, 3))
469     family = [M, M @ M - M, np.eye(3) + 0.3 * M @ M @ M]
470     f = [np.cos, lambda t: t, lambda t: np.exp(-t)]
471     F = [np.sin, lambda t: t**2 / 2, lambda t: -np.exp(-t)]  # antiderivatives of f
472
473     def A5(t):
474         return sum(f_j(t) * A_j for f_j, A_j in zip(f, family))
475
476     def Phi5_exp(t, tau):
477         return expm(sum((F_j(t) - F_j(tau)) * A_j for F_j, A_j in zip(F, family)))
478
479     def Phi5_factored(t, tau):
480         factors = [expm((F_j(t) - F_j(tau)) * A_j) for F_j, A_j in zip(F, family)]
481         return reduce(np.matmul, factors)
482
483     report(
484         "s05b",
485         "commuting A_j (polynomials in a random 3x3 M), f = cos(t), t, e^(-t):\n"
486         "max relative error vs numerical STM:\n"
487         f"  exp(sum_j F_j A_j)       {stm_error(A5, Phi5_exp):.1e}\n"
488         f"  prod_j exp(F_j A_j)      {stm_error(A5, Phi5_factored):.1e}",
489     )
490
491     # c) A(t) = cos(3t) I + sin(3t) J
492     def A5c(t):
493         return np.array(
494             [[np.cos(3 * t), np.sin(3 * t)], [-np.sin(3 * t), np.cos(3 * t)]]
495         )
496
497     def Phi5c(t, tau):
498         theta = (np.cos(3 * tau) - np.cos(3 * t)) / 3
499         rotation = np.array(
500             [[np.cos(theta), np.sin(theta)], [-np.sin(theta), np.cos(theta)]]
501         )
502         return np.exp((np.sin(3 * t) - np.sin(3 * tau)) / 3) * rotation
503
504     samples = rng.uniform(-5, 5, size=(20, 2))
```

```
    commutator = max(
        np.abs(A5c(t1) @ A5c(t2) - A5c(t2) @ A5c(t1)).max() for t1, t2 in samples
    )
    report(
        "s05c",
        f"max |A(t1) A(t2) - A(t2) A(t1)| over 20 random pairs = {commutator:.1e}\n"
        "max relative error of Phi(t, tau) vs numerical STM = "
        f"{stm_error(A5c, Phi5c):.1e}",
    )

    #########
    # p06 #
    #########
    heading("p06")

    def A6a(t):
        return np.diag([t, np.cos(t)])

    def Phi6a(t, tau):
        return np.diag([np.exp((t**2 - tau**2) / 2), np.exp(np.sin(t) - np.sin(tau))])

    def A6b(t):
        return np.cos(2 * t) * np.array([[1.0, 1.0], [0.0, 0.0]])

    def Phi6b(t, tau):
        g = np.exp((np.sin(2 * t) - np.sin(2 * tau)) / 2)
        return np.array([[g, g - 1], [0.0, 1.0]])

    def A6c(t):
        return np.array([[-2.0, np.sin(t)], [0.0, -1.0]])

    def Phi6c(t, tau):
        fast, slow = np.exp(-2 * (t - tau)), np.exp(-(t - tau))
        coupling = (
            slow * (np.sin(t) - np.cos(t)) - fast * (np.sin(tau) - np.cos(tau))
        ) / 2
        return np.array([[fast, coupling], [0.0, slow]])

    # exp of the integral of A: not the STM, since A(t) A(s) != A(s) A(t)
    def Phi6c_naive(t, tau):
        return expm(
            np.array([[-2 * (t - tau), np.cos(tau) - np.cos(t)], [0, -(t - tau)]])
        )

    A6d = np.array([[-3.0, -1.0], [-1.0, -3.0]])
    eigvals6, V = np.linalg.eigh(A6d)
```

```
    W = np.linalg.inv(V)  # rows are the reciprocal basis

    def Phi6d(t, tau):
        return sum(
            np.exp(lam * (t - tau)) * np.outer(V[:, k], W[k])
            for k, lam in enumerate(eigvals6)
        )

    def Phi6d_hand(t, tau):
        d = t - tau
        return (
            np.exp(-2 * d) * np.array([[1, -1], [-1, 1]])
            + np.exp(-4 * d) * np.array([[1, 1], [1, 1]])
        ) / 2

    commutator = np.abs(A6c(1.0) @ A6c(2.0) - A6c(2.0) @ A6c(1.0)).max()
    naive = max(
        np.abs(Phi6c_naive(t, 0.0) - Phi6c(t, 0.0)).max() for t in np.linspace(0, 3, 31)
    )
    dyadic = max(
        (np.abs(Phi6d(t, 0.0) - expm(A6d * t)) / np.abs(expm(A6d * t)).max()).max()
        for t in np.linspace(-3, 3, 61)
    )
    report(
        "s06",
        "max relative error of Phi(t, tau) vs numerical STM:\n"
        f"  a) {stm_error(A6a, Phi6a):.1e}\n"
        f"  b) {stm_error(A6b, Phi6b):.1e}\n"
        f"  c) {stm_error(A6c, Phi6c):.1e}\n"
        f"  d) {stm_error(lambda t: A6d, Phi6d_hand):.1e}\n"
        f"c) |A(1) A(2) - A(2) A(1)| = {commutator:.3f}\n"
        f"c) max |exp(int A) - Phi| = {naive:.3f}  (exp(int A) is not the STM)\n"
        f"d) eigenvalues = {show(eigvals6)}\n"
        f"d) max relative |dyadic sum - e^(At)| = {dyadic:.1e}",
    )

    #########
    # p07 #
    #########
    heading("p07")

    def A7(t):
        s, c = np.sin(t), np.cos(t)
        return np.array(
            [[-1 + 1.5 * c * c, 1 - 1.5 * s * c], [-1 - 1.5 * s * c, -1 + 1.5 * s * s]]
        )
```

```
597
598     def Psi7(t):
599         s, c = np.sin(t), np.cos(t)
600         return np.array(
601             [[np.exp(-t) * s, np.exp(t / 2) * c], [np.exp(-t) * c, -np.exp(t / 2) * s]]
602         )
603
604     def dPsi7(t):
605         s, c = np.sin(t), np.cos(t)
606         return np.array(
607             [
608                 [np.exp(-t) * (c - s), np.exp(t / 2) * (c / 2 - s)],
609                 [np.exp(-t) * (-s - c), np.exp(t / 2) * (-s / 2 - c)],
610             ]
611         )
612
613     def Phi7(t, tau):
614         return Psi7(t) @ np.linalg.inv(Psi7(tau))
615
616     # a) frozen-time eigenvalues
617     t = np.linspace(0, 2 * np.pi, 721)
618     frozen = np.array([np.linalg.eigvals(A7(tk)) for tk in t])
619     assert np.allclose(frozen.real, -0.25) and np.allclose(
620         np.abs(frozen.imag), np.sqrt(7) / 4
621     )
622     report(
623         "s07a",
624         f"eigenvalues of A(t) on a 721-point grid over [0, 2 pi]:\n"
625         f"  real part in [{frozen.real.min():.6f}, {frozen.real.max():.6f}]\n"
626         f"  |imaginary part| in [{np.abs(frozen.imag).min():.6f}, "
627         f"{np.abs(frozen.imag).max():.6f}],  sqrt(7)/4 = {np.sqrt(7) / 4:.6f}",
628     )
629
630     # b) fundamental solutions
631     t = np.linspace(-3, 6, 181)
632     residual = max(
633         (np.abs(dPsi7(tk) - A7(tk) @ Psi7(tk)) / np.maximum(1, np.abs(Psi7(tk)))).max()
634         for tk in t
635     )
636     det_error = max(abs(np.linalg.det(Psi7(tk)) + np.exp(-tk / 2)) for tk in t)
637     assert residual < 1e-12 and det_error < 1e-9
638     report(
639         "s07b",
640         f"max relative |dPsi/dt - A(t) Psi(t)| on [-3, 6] = {residual:.1e}\n"
641         f"max |det Psi(t) + e^(-t/2)| on [-3, 6] = {det_error:.1e}",
642     )
```

```
643
644     # c) STM, monodromy matrix, and divergence
645     monodromy = stm_numeric(A7, 0.0, np.array([0.0, np.pi]))[-1]
646     multipliers = np.sort(np.linalg.eigvals(monodromy).real)
647     assert np.allclose(multipliers, [-np.exp(np.pi / 2), -np.exp(-np.pi)])
648     report(
649         "s07c",
650         "max relative error of Phi(t, tau) vs numerical STM = "
651         f"{stm_error(A7, Phi7):.1e}\n"
652         f"monodromy matrix Phi(pi, 0) =\n{show(monodromy)}\n"
653         f"Floquet multipliers = {show(multipliers)}\n"
654         f"-e^(pi/2) = {-np.exp(np.pi / 2):.4f}, -e^(-pi) = {-np.exp(-np.pi):.4f}",
655     )
656
657     t = np.linspace(0, 10, 501)
658     initial_states = [
659         (r"$\mathbf{x}_0 = [1,\ 0]^T = \psi_2(0)$", [1.0, 0.0], BLUE, "-"),
660         (r"$\mathbf{x}_0 = [0,\ 1]^T = \psi_1(0)$", [0.0, 1.0], ORANGE, "--"),
661         (r"$\mathbf{x}_0 = [1,\ 1]^T/\sqrt{2}$", [2**-0.5, 2**-0.5], AQUA, "-."),
662     ]
663     fig, ax = plt.subplots(figsize=(6.4, 3.4))
664     for label, x0, color, ls in initial_states:
665         sol = solve_ivp(
666             lambda s, x: A7(s) @ x,
667             (0, 10),
668             x0,
669             method="DOP853",
670             t_eval=t,
671             rtol=1e-11,
672             atol=1e-13,
673         )
674         exact = np.array([Phi7(tk, 0.0) @ x0 for tk in t])
675         assert (np.abs(sol.y.T - exact) / np.maximum(1, np.abs(exact))).max() < 1e-7
676         ax.semilogy(t, np.linalg.norm(sol.y, axis=0), color=color, ls=ls, label=label)
677     ax.semilogy(
678         t,
679         np.exp(-t / 4),
680         color=MUTED,
681         ls=":",
682         label=r"frozen-eigenvalue decay $e^{-t/4}$",
683     )
684     ax.set(xlabel=r"Time $t$", ylabel=r"$\|\mathbf{x}(t)\|$")
685     ax.grid(True, which="major")
686     ax.legend(fontsize=8)
687     save(fig, "s07c")
688
```

```
689
690 if __name__ == "__main__":
691     main()
```